

مدل‌های Sequence-to-Sequence و مکانیزم توجه

مدل‌های Sequence-to-Sequence در یادگیری عمیق در ترجمه‌ی ماشینی، خلاصه‌سازی متن و تولید کپشن برای تصاویر کاربرد دارند.

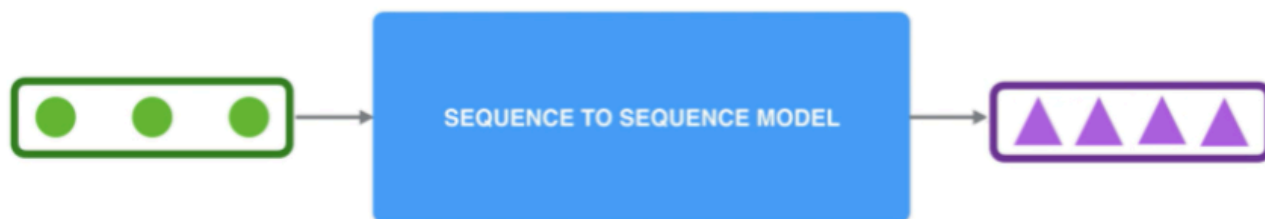
گوگل ترنسلیت از اواخر سال ۲۰۱۶ شروع به استفاده از چنین مدلی در سیستم‌های تولیدی خود کرد.

این مدل‌ها در دو مقاله‌ی (Sutskever et al., 2014, Cho et al., 2014) تشریح شده‌اند.

برای پیاده‌سازی این مدل‌ها و درک کامل آنها باید با مجموعه‌ای از مفاهیم که به تدریج بر روی یکدیگر بنا می‌شوند آشنا شد که در این بخش برخی از این مفاهیم را با استفاده از نمایش‌های بصری ساده‌تر بیان می‌کنیم. برای دنبال کردن این مبحث، نیاز به درک اولیه از یادگیری عمیق خواهید داشت.

مروری بر مدل‌های Sequence-to-Sequence

یک مدل Sequence-to-Sequence مدلی است که یک دنباله از آیتم‌ها (مانند کلمات، حروف، یا ویژگی‌های یک تصویر) را دریافت کرده و یک دنباله‌ی جدید از آیتم‌ها را تولید می‌کند. یک مدل آموزش‌دیده به شکل زیر عمل می‌کند:



در کاربرد ترجمه‌ی ماشینی (Neural Machine Translation)، دنباله‌ی ورودی شامل یک سری کلمات است که به صورت ترتیبی پردازش می‌شوند. خروجی نیز یک دنباله از کلمات است که معادل ترجمه‌ی متن ورودی است.

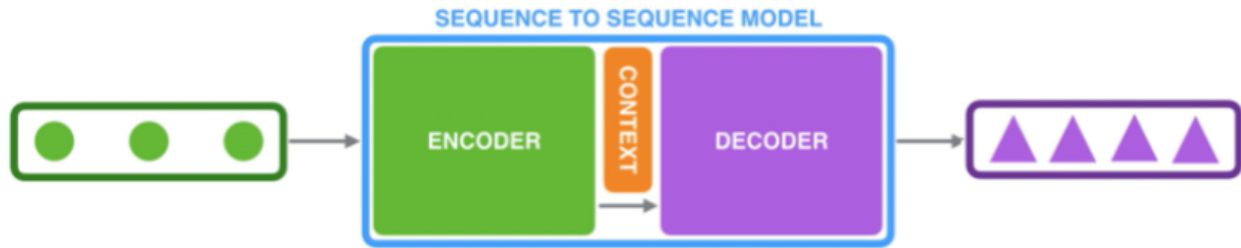
ساختار مدل Sequence-to-Sequence: رمزگذار و رمزگشا

در قلب این مدل‌ها، دو بخش اصلی وجود دارد:

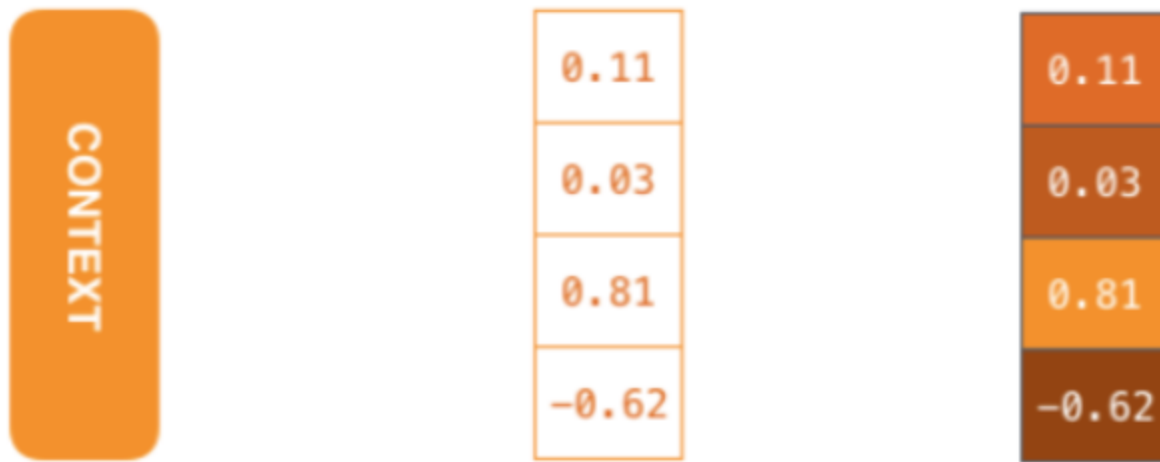
1. **رمزگذار (Encoder):** پردازش دنباله‌ی ورودی و استخراج اطلاعات مرتبط در قالب یک بردار متنی که به آن بردار متن (Context Vector) گفته می‌شود.
2. **رمزگشا (Decoder):** دریافت بردار متن و تولید دنباله‌ی خروجی، واحد به واحد.

در مدل‌های ترجمه‌ی ماشینی، بردار متن یک آرایه از اعداد است که رمزگذار و رمزگشا هر دو شبکه‌های عصبی بازگشتی (RNN) هستند.

شکل زیر نمایش ساده‌ای از این ساختار است:



اندازه‌ی بردار متن هنگام طراحی مدل تعیین می‌شود. در مدل‌های واقعی، اندازه‌ی آن معمولاً ۲۵۶، ۵۱۲ یا ۱۰۲۴ است. این بردار را می‌توان در شکل زیر دید که در آن مقادیر بیشتر با رنگ‌های پررنگ‌تری نمایش داده شده است.



بردارهای تعبیه‌شده (امبدینگ) و ورودی شبکه‌های عصبی بازگشتی

همان طور که در شکل‌های بعدی ملاحظه می‌کنیم، شبکه‌های عصبی بازگشتی (RNN) در هر مرحله دو ورودی دریافت می‌کنند:

1. یک ورودی جدید (در رمزگذار، این ورودی یک کلمه از جمله‌ی ورودی است).
2. حالت پنهان (Hidden State) که اطلاعات مراحل قبلی را ذخیره می‌کند.

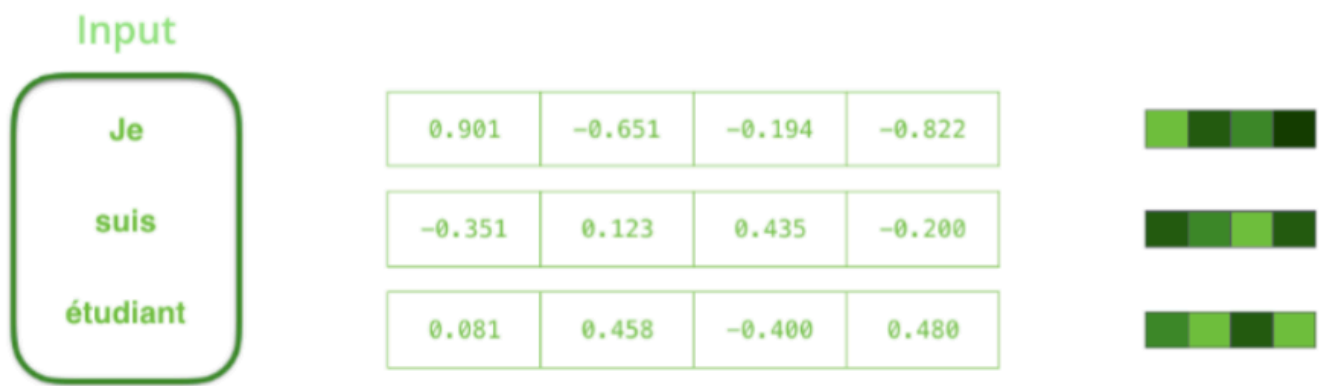
برای استفاده از این اطلاعات، کلمات باید به بردارهای عددی تبدیل شوند.

این کار توسط روش‌های تعبیه‌ی کلمات (Word Embedding) انجام می‌شود که ارتباط معنایی بین کلمات را حفظ می‌کنند.

برای مثال:

$$\text{king} - \text{man} + \text{woman} = \text{queen}$$

شکل زیر نمایش ساده‌ای از این فرآیند است:



اندازه‌ی این بردارها معمولاً ۲۰۰ یا ۳۰۰ بُعدی است و معمولاً از اعداد float تشکیل شده اند.

حالات پنهان در رمزگذار و رمزگشا

در شکل زیر، هر پالس در رمزگذار یا رمزگشا نشان‌دهنده‌ی یک مرحله‌ی پردازش در RNN است.

در هر گام زمانی (Time Step)، یکی از شبکه‌های عصبی بازگشتی (RNN) داده‌ی خود را پردازش کرده

و خروجی مربوط به همان مرحله را تولید می‌کند.

از آنجایی که هم رمزگذار و هم رمزگشا مبتنی بر RNN هستند، در هر مرحله، یکی از این RNN‌ها ورودی خود را پردازش کرده و حالت پنهان خود را به‌روزرسانی می‌کند. این به‌روزرسانی بر اساس ورودی فعلی و اطلاعات قبلی که دیده است انجام می‌شود.

Recurrent Neural Network

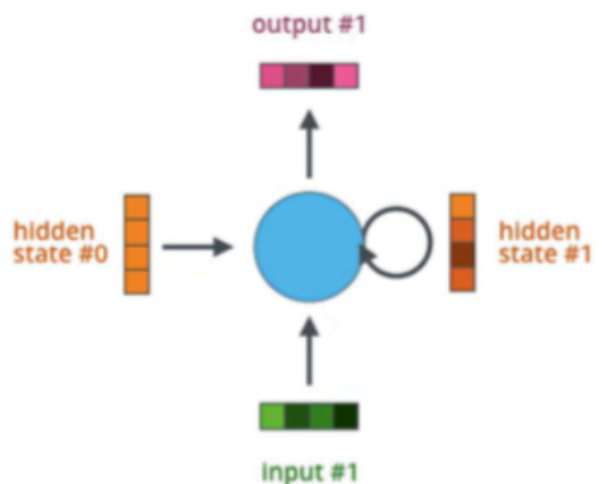
Time step #1:

An RNN takes two input vectors:



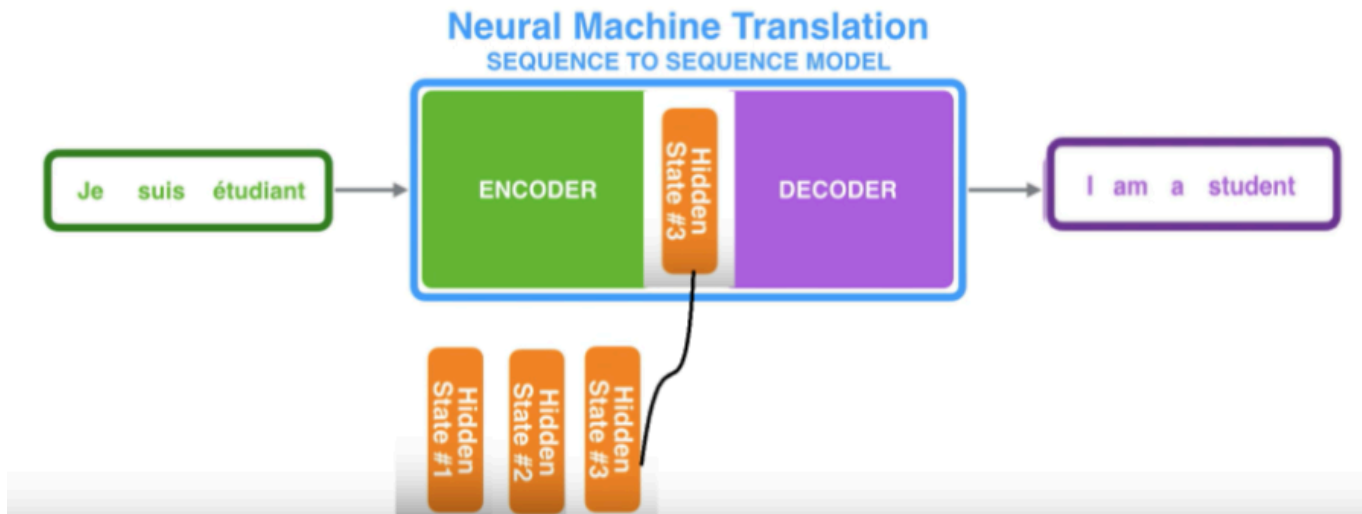
Processes them

Then produces two output vectors:



حال، بیایید حالات پنهان (Hidden States) رمزگذار را بررسی کنیم.

به شکل زیر توجه کنید. مشاهده می‌کنیم که آخرین حالت پنهان رمزگذار درواقع همان بردار متن (Context Vector) است که به رمزگشا ارسال می‌شود.



این فرآیند نشان می‌دهد که تمام اطلاعات جمع‌آوری‌شده در رمزگذار در قالب بردار متن به رمزگشا انتقال می‌یابد.

این موضوع در مدل‌های Sequence-to-Sequence اهمیت بالایی دارد، زیرا رمزگشا از این اطلاعات برای تولید خروجی مرحله به مرحله استفاده می‌کند.

نمایش گسترش‌یافته‌ی مدل

همان‌طور که در رمزگذار مشاهده کردیم، رمزگشا نیز یک حالت پنهان (Hidden State) را حفظ می‌کند و آن را در هر گام زمانی به گام بعدی منتقل می‌کند.

با این حال، در تصاویر قبلی این بخش را به نمایش نگذاشتیم، زیرا در حال تمرکز بر اجزای اصلی مدل بودیم.

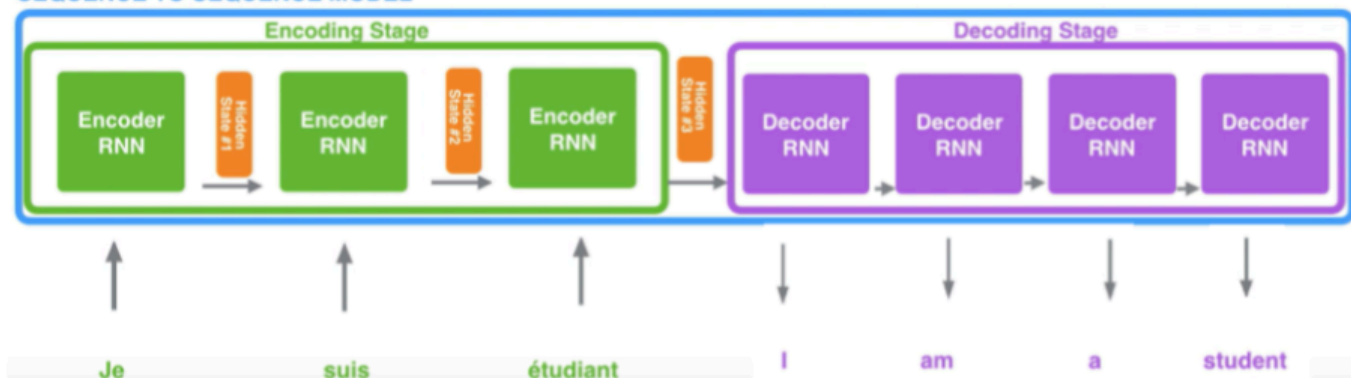
حال، بیایید به روش دیگری برای تجسم مدل Seq2Seq نگاه کنیم. استفاده از یک نمایش گسترش‌یافته (Unrolled View) می‌تواند درک مدل را ساده‌تر کند.

در این روش، به جای نمایش یک نمونه از رمزگشا، چندین نسخه از آن را برای هر گام زمانی نمایش می‌دهیم.

این نمایش به ما امکان می‌دهد که ورودی‌ها و خروجی‌های هر گام زمانی را به طور مشخص‌تر بررسی کنیم.

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL



همان‌طور که در شکل بالا مشاهده می‌شود، در هر گام زمانی، یک کپی از رمزگشا پردازش‌های مربوط به ورودی آن مرحله را انجام داده و خروجی را تولید می‌کند.

این فرآیند به صورت تکراری تا پایان دنباله خروجی ادامه می‌یابد.

محدودیت بردار متن و ظهور مکانیزم توجه

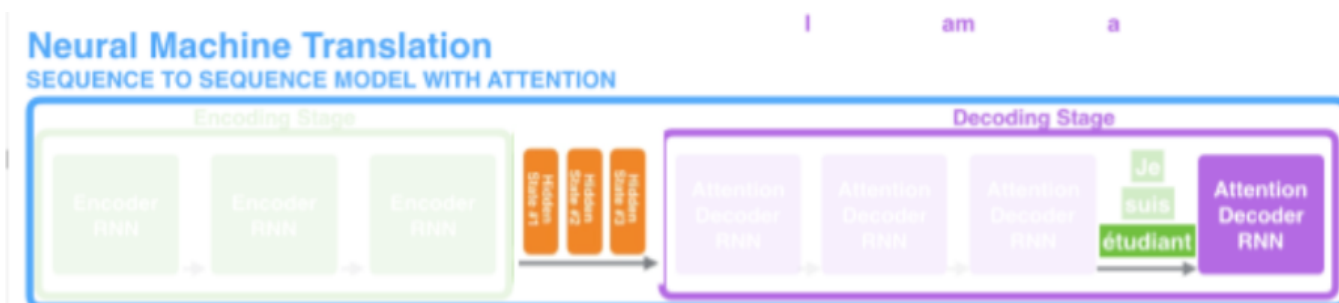
بردار متن (Context Vector) یک نقطه‌ی ضعف مهم در مدل‌های Sequence-to-Sequence مثل RNN دارد:

- هنگام ترجمه‌ی جملات بلند، مدل دچار مشکل می‌شود.
- کل جمله‌ی ورودی به یک بردار تبدیل شده و برخی از اطلاعات آن از بین می‌رود.

برای حل این مشکل، مکانیزم توجه (Attention) توسط Bahdanau et al., 2014 و Luong et al., 2015 پیشنهاد شد.

- مدل‌های مبتنی بر توجه به جای استفاده از تنها یک بردار متن، از همه‌ی حالات پنهان رمزگذار استفاده می‌کنند.
- در هر مرحله از تولید خروجی، رمزگشا بر بخش‌های مرتبط از ورودی تمرکز می‌کند.

شکل زیر نمایشگر این فرآیند است:



مکانیزم توجه چگونه کار می‌کند؟

در مدل‌های بدون توجه، رمزگشا تنها از آخرین حالت پنهان رمزگذار استفاده می‌کند.

اما در مدل‌های با توجه:

1. تمام حالات پنهان رمزگذار به رمزگشا ارسال می‌شوند.
 2. رمزگشا در هر مرحله‌ی تولید خروجی:
 - هر کلمه‌ی ورودی را امتیازدهی می‌کند.
 - ضریب اهمیت هر کلمه را با استفاده از تابع Softmax محاسبه می‌کند.
 - بردار متن جدیدی ایجاد می‌کند که اطلاعات کلیدی را تقویت و اطلاعات غیرضروری را کاهش می‌دهد.
- این فرآیند در هر مرحله از تولید خروجی تکرار می‌شود.
- شکل زیر نمایش ساده‌ای از نحوه‌ی عملکرد مکانیزم توجه را نشان می‌دهد:



جمع‌بندی

مدل‌های Sequence-to-Sequence روش قدرتمندی برای ترجمه‌ی ماشینی و خلاصه‌سازی متن هستند. با این حال، بردار متن (context vector) یک محدودیت جدی ایجاد می‌کرد که مکانیزم توجه آن را حل کرد. این مدل‌ها در حال حاضر اساس مدل‌های زبانی مدرن مانند Transformer و BERT را تشکیل می‌دهند.

نگاهی به مدل ترنسفورمر

در بخش قبلی، به مفهوم توجه (Attention) پرداختیم که روشی کلیدی در مدل‌های یادگیری عمیق امروزی محسوب می‌شود.

این مفهوم توانست عملکرد ترجمه ماشینی عصبی را بهبود بخشد. در این بخش، به بررسی مدل ترنسفورمر (Transformer) خواهیم پرداخت؛ مدلی که از سازوکار توجه (attention) استفاده می‌کند تا سرعت آموزش مدل‌های یادگیری عمیق را افزایش دهد. مدل ترنسفورمر در مقاله‌ی "Attention is All You Need" معرفی شده است. این مدل در برخی وظایف خاص از مدل ترجمه ماشینی عصبی گوگل (Google Neural Machine Translation) بهتر عمل می‌کند.

اما مهم‌ترین مزیت آن قابلیت موازی‌سازی است که موجب افزایش بهره‌وری پردازشی می‌شود. گوگل کلود توصیه کرده است که مدل ترنسفورمر به‌عنوان مدل مرجع برای استفاده از Cloud TPU به کار رود. در ادامه، سعی داریم مدل را به بخش‌های کوچک‌تر تقسیم کرده و نحوه عملکرد آن را بررسی کنیم.

نمای کلی مدل

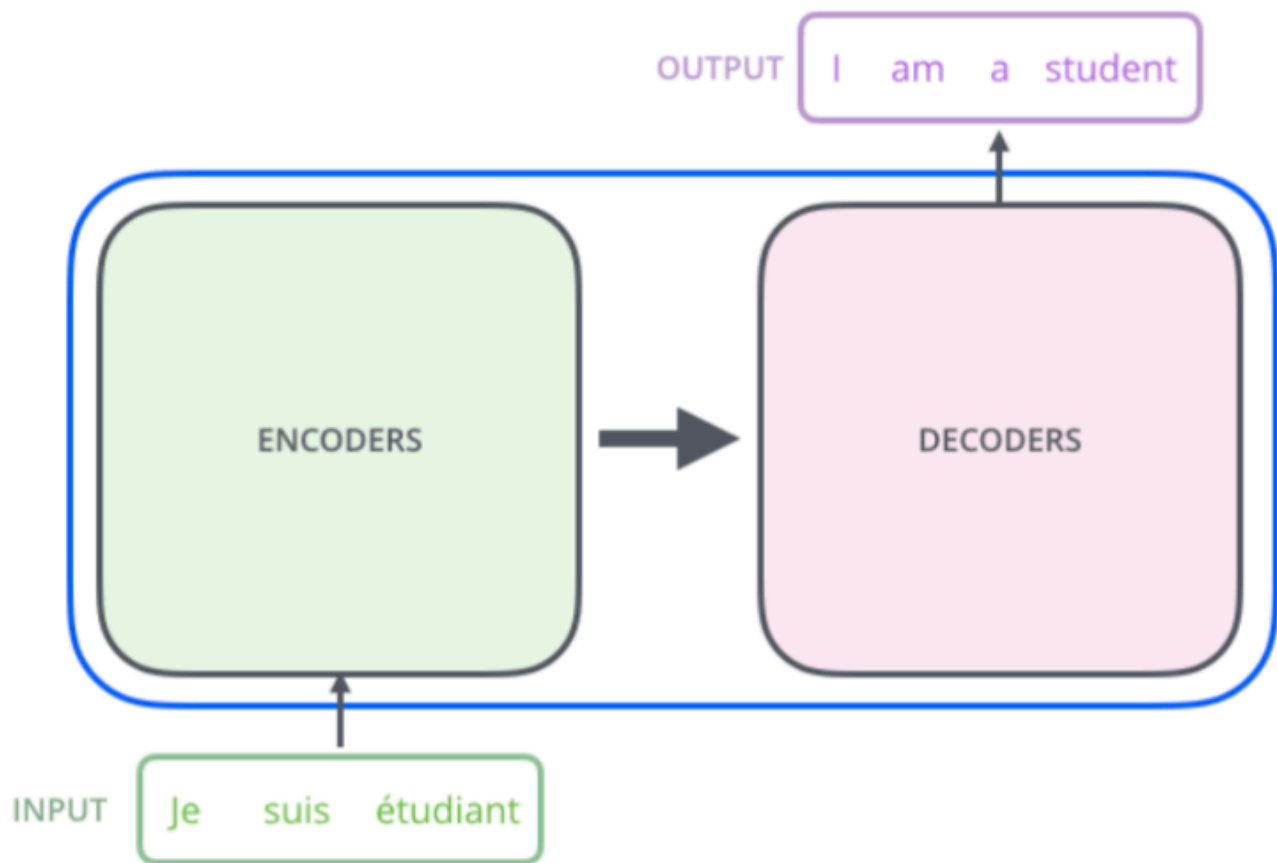
برای شروع، مدل را به‌عنوان یک جعبه سیاه در نظر می‌گیریم. در یک کاربرد ترجمه ماشینی، این مدل یک جمله در یک زبان را دریافت کرده و ترجمه آن را در زبانی دیگر تولید می‌کند.



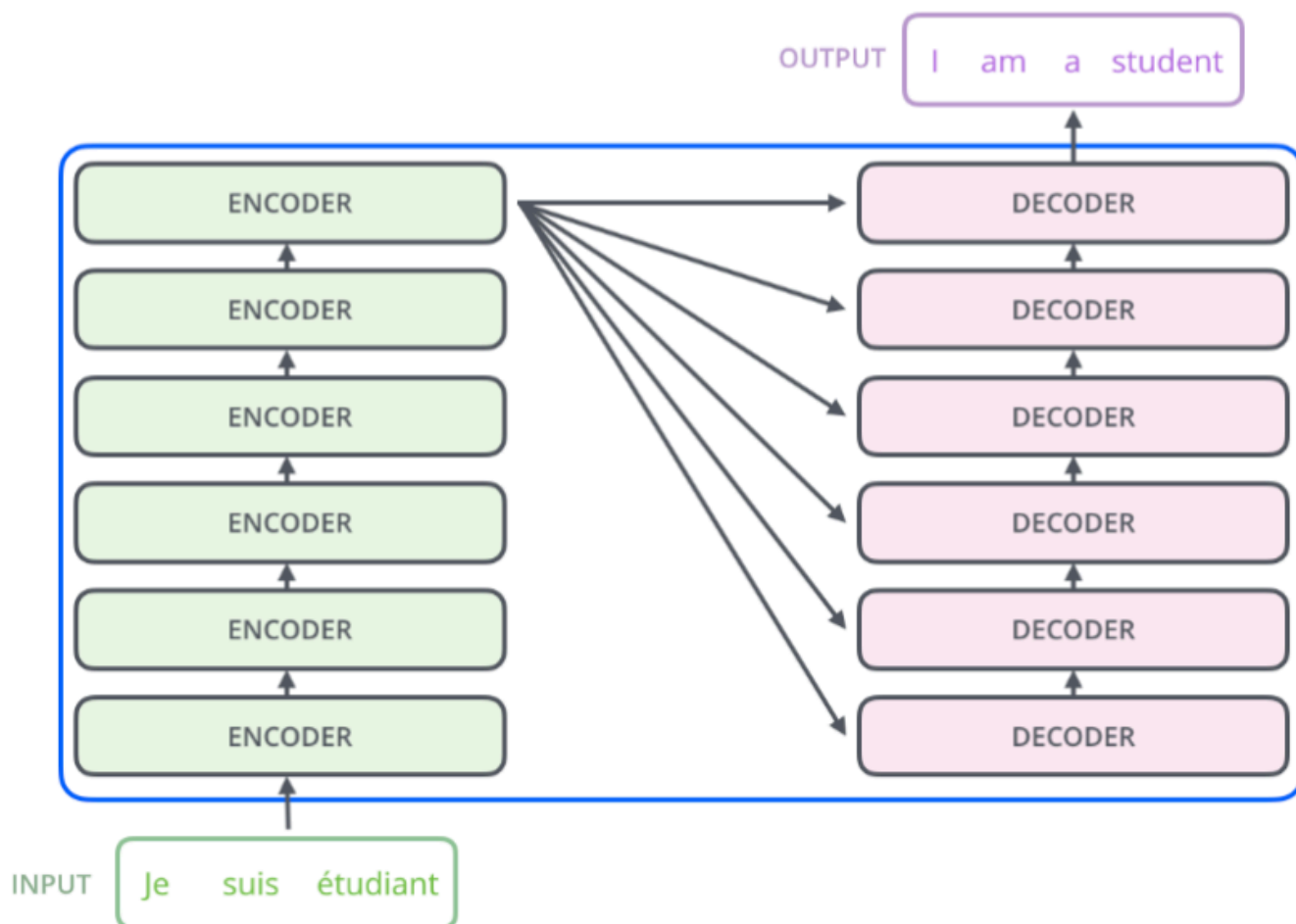
ساختار مدل

با بررسی ساختار داخلی مدل، متوجه دو مؤلفه اصلی می‌شویم:

- **رمزگذار (Encoder):** مسئول پردازش متن ورودی و استخراج ویژگی‌های معنایی آن.
- **رمزگشا (Decoder):** وظیفه تولید متن خروجی را بر عهده دارد.



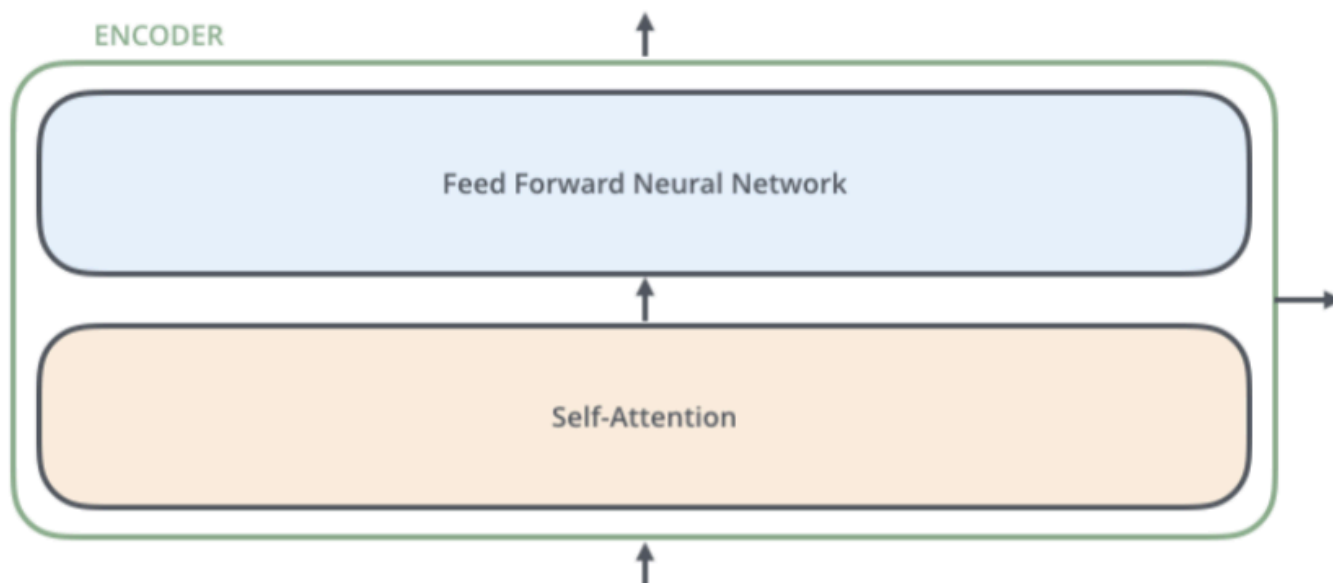
در معماری پیشنهادی مقاله، رمزگذار از شش لایه مشابه تشکیل شده است (این مقدار قابل تنظیم است و می‌توان تعداد بیشتری از رمزگذارها را به کار برد). رمزگشا نیز از همین تعداد لایه تشکیل شده است.



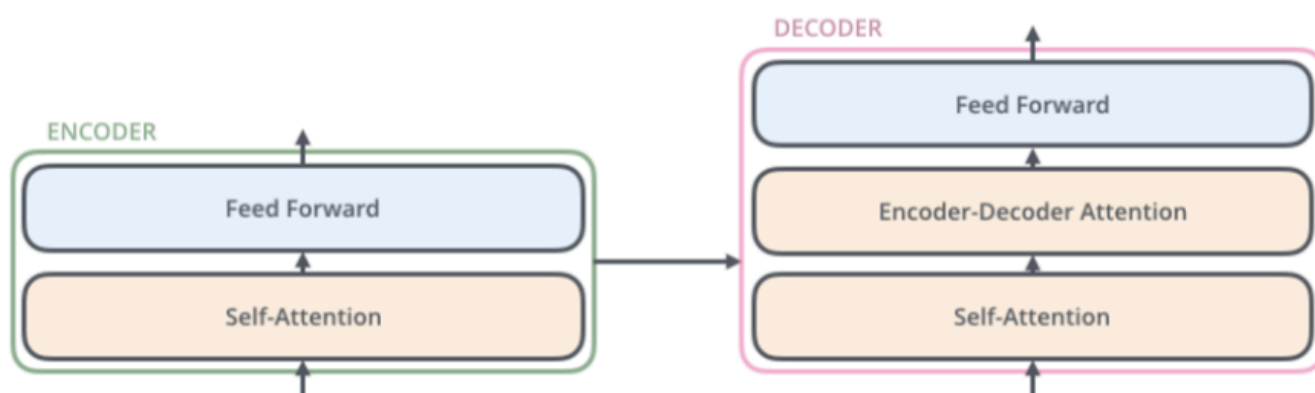
اجزای هر یک از بلوک های رمزگذار (Encoder) و رمزگشا (Decoder)

همان طور که در تصویر زیر ملاحظه می کنید هر رمزگذار از دو زیرلایه تشکیل شده است:

1. **لایه توجه به خود (Self-Attention):** این لایه باعث می شود که مدل درک بهتری از روابط بین کلمات متن ورودی داشته باشد.
2. **شبکه عصبی کاملاً متصل (Feed-Forward Neural Network):** این لایه در هر موقعیت زمانی (به ازای هر توکن) به طور مستقل پردازش را انجام می دهد.



رمزگشا علاوه بر این دو لایه، یک لایه توجه اضافی نیز دارد که به آن کمک می‌کند بر بخش‌های مهم‌تر جمله ورودی تمرکز کند (مشابه مکانسزم توجه در مدل‌های Seq2Seq).



ورود تنسورها به مدل

اکنون که اجزای اصلی مدل را شناختیم، به بررسی نحوه پردازش بردارها (Vectors) و تنسورها (Tensors) در مدل می‌پردازیم.

همان‌طور که در بسیاری از کاربردهای پردازش زبان طبیعی (NLP) دیده‌ایم، ابتدا کلمات ورودی را به بردارهای امبدینگ (Word Embeddings) تبدیل می‌کنیم.

فرآیند امبدینگ و عبور از لایه‌های رمزگذار

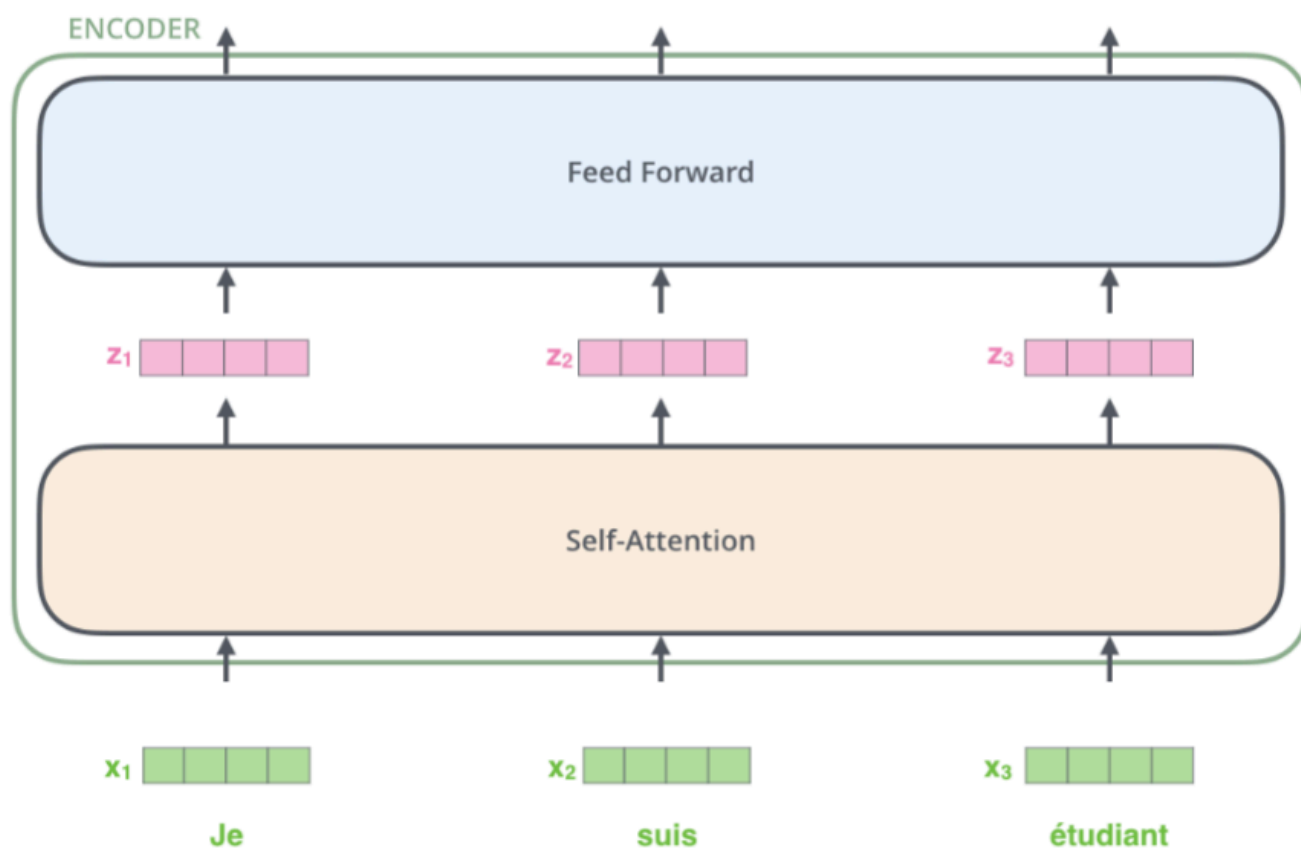
بردار امبدینگ کلمات تنها در اولین رمزگذار (پایین‌ترین رمزگذار در پشته) انجام می‌شود.

در تمامی رمزگذارهای بعدی، ورودی هر رمزگذار، خروجی رمزگذار قبلی خواهد بود.

همه رمزگذارها یک ساختار یکسان دارند و هر کدام یک لیست از بردارها را دریافت می‌کنند که هر بردار دارای ابعاد ۵۱۲ است.

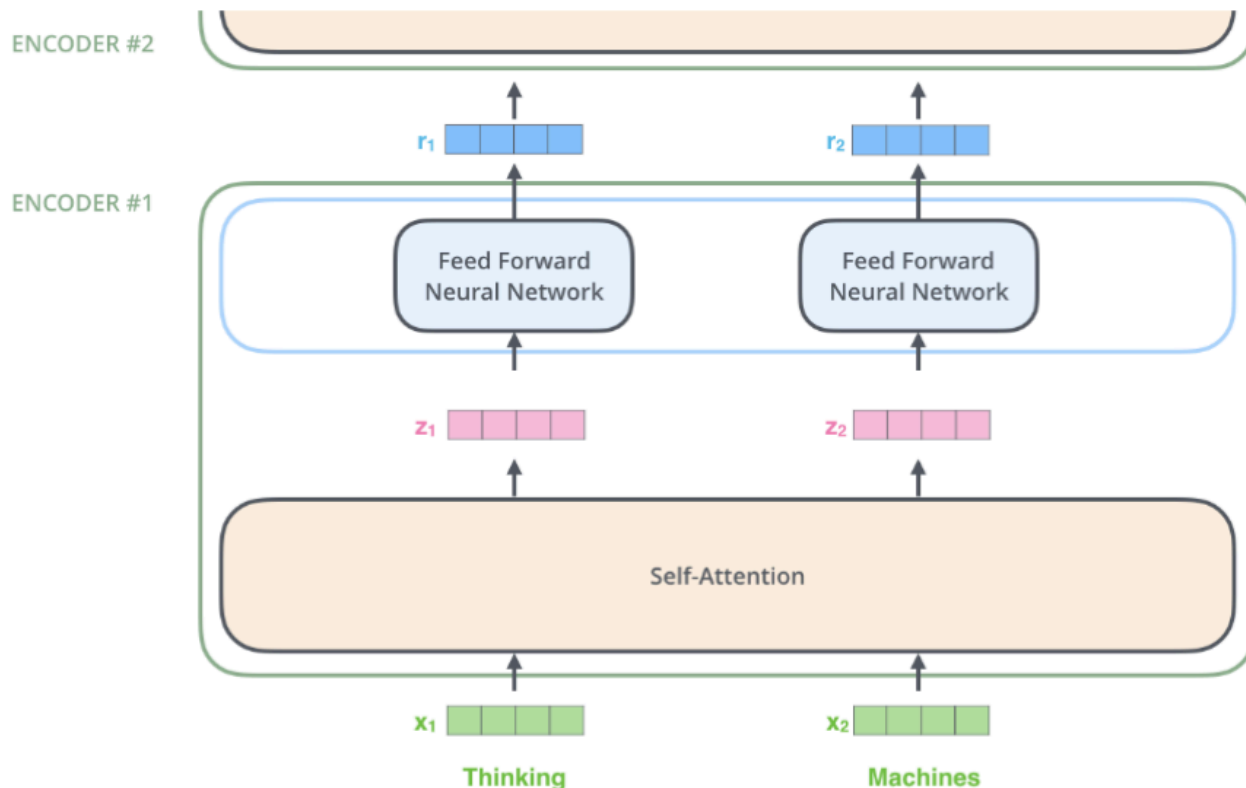
در اولین رمزگذار، این بردارها همان بردارهای امبدینگ کلمات هستند، اما در رمزگذارهای بالاتر، این بردارها نتیجه‌ی پردازش رمزگذار قبلی خواهند بود.

تعداد این بردارها (طول لیست ورودی) به‌عنوان یک اَبرپارامتر قابل تنظیم است. به‌طور معمول، طول این لیست برابر با طولانی‌ترین جمله در مجموعه داده‌ی آموزشی انتخاب می‌شود.



پس از انجام عملیات امبدینگ کلمات، هر یک از این بردارها از طریق دو لایه اصلی رمزگذار عبور می‌کنند:

- **لایه توجه به خود (Self-Attention):** این لایه اطلاعات مرتبط با سایر کلمات متن را استخراج می‌کند.
- **شبکه‌ی عصبی کاملاً متصل (Feed-Forward Neural Network):** این لایه ویژگی‌های پردازش‌شده را تقویت کرده و خروجی جدیدی تولید می‌کند.



این فرآیند در هر رمزگذار به صورت مستقل انجام می شود و در نهایت خروجی هر رمزگذار به رمزگذار بعدی منتقل می شود تا اطلاعات مورد نیاز در سطح های بالاتر مدل به طور عمیق تر پردازش شود.

یکی از ویژگی های مهم ترنسفورمر این است که هر کلمه در مسیر جداگانه ای پردازش می شود و فقط در لایه توجه به خود، تعامل بین مسیرها رخ می دهد. اما در لایه شبکه ی عصبی کاملاً متصل (feed forward neural network layer)، این مسیرها به طور مستقل پردازش می شوند که این امکان موازی سازی محاسبات را فراهم می آورد.

درک مفهوم توجه به خود (Self-Attention)

شاید با شنیدن واژه ی توجه به خود (Self-Attention) تصور کنید که این یک مفهوم آشنا و عمومی در یادگیری عمیق است. اما در واقع تا قبل از انتشار مقاله ی مشهور "Attention is All You Need" این مفهوم چندان شناخته شده نبود.

بیایید با یک مثال ساده، عملکرد آن را بررسی کنیم.

جمله ی زیر را در نظر بگیرید که می خواهیم آن را ترجمه کنیم:

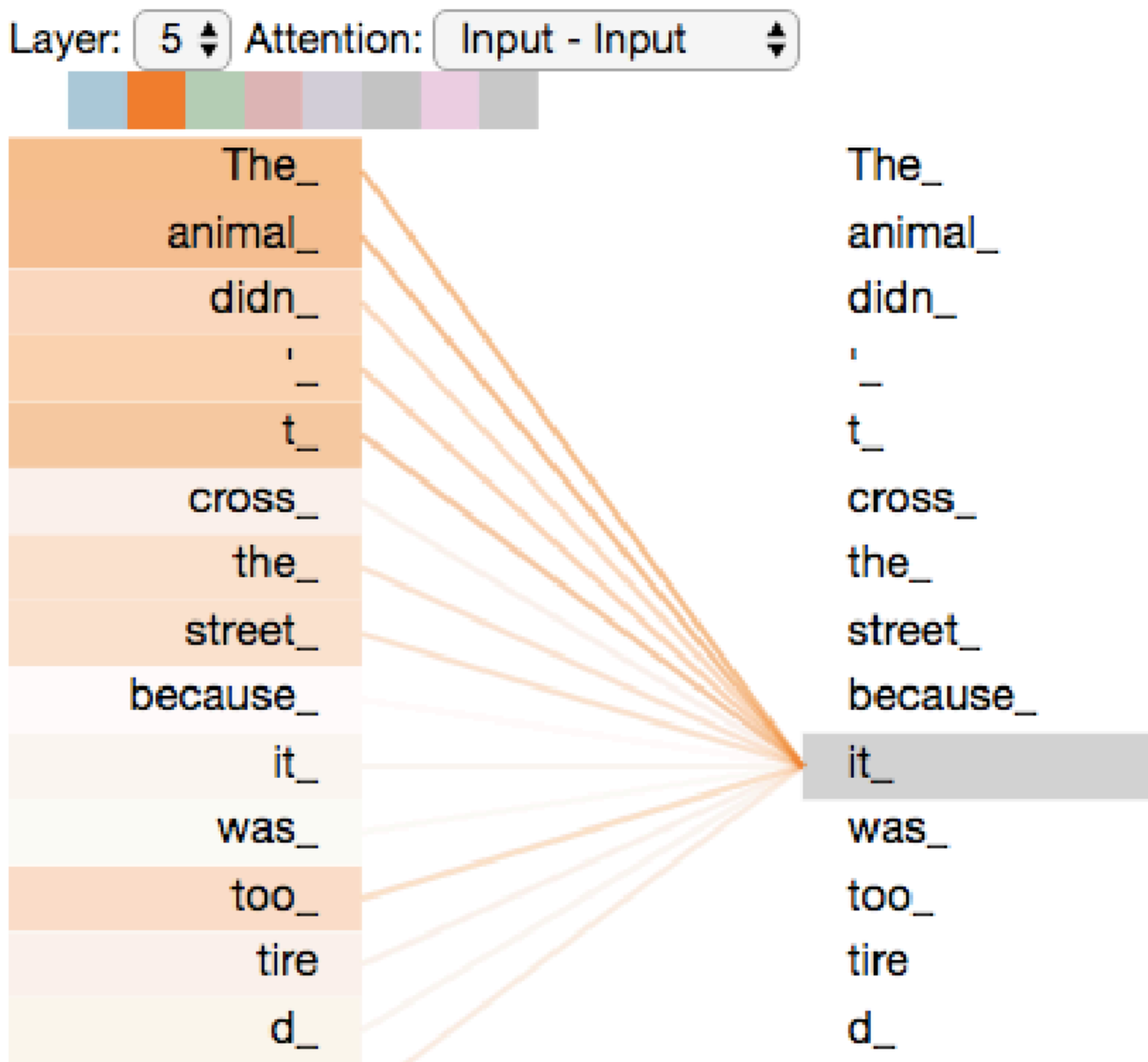
"The animal didn't cross the street because it was too tired."

حال، سوال مهم این است که "it" در این جمله به چه چیزی اشاره دارد؟

آیا منظور خیابان (street) است یا حیوان (animal)؟

برای یک انسان، پاسخ به این پرسش بسیار ساده است، اما برای یک مدل زبانی چالش برانگیز می باشد. اینجاست که مکانیزم توجه به خود وارد عمل می شود!

در حین پردازش کلمه‌ی "it"، مکانیزم توجه به خود به مدل کمک می‌کند تا ارتباط این کلمه با "animal" را درک کند.



مکانیزم توجه به خود به مدل کمک می‌کند تا ارتباط واژه‌های متن را به‌درستی برقرار کند. در این مثال، توجه به خود باعث می‌شود مدل درک کند که "it" به "animal" اشاره دارد.

همان‌طور که مدل هر کلمه را پردازش می‌کند، مکانیزم توجه به خود به آن اجازه می‌دهد تا به سایر کلمات موجود در جمله نگاه کند و سرخ‌هایی برای بهبود رمزگذاری آن کلمه پیدا کند.

اگر با شبکه‌های عصبی بازگشتی (RNN) آشنا باشید، می‌دانید که در RNN، وضعیت پنهان (Hidden State) به مدل کمک می‌کند تا اطلاعات کلمات قبلی را حفظ کند و آن‌ها را در پردازش کلمات جدید لحاظ کند. اما در مدل ترنسفورمر، توجه به خود این نقش را ایفا می‌کند و به مدل اجازه می‌دهد که اطلاعات سایر کلمات مهم را در پردازش هر کلمه لحاظ کند.

به عبارت دیگر، مدل توجه به خود (self attention) درک و ارتباط میان واژه‌ها را در فرآیند رمزگذاری بهبود می‌بخشد.

محاسبه‌ی توجه به خود (Self-Attention) به صورت دقیق

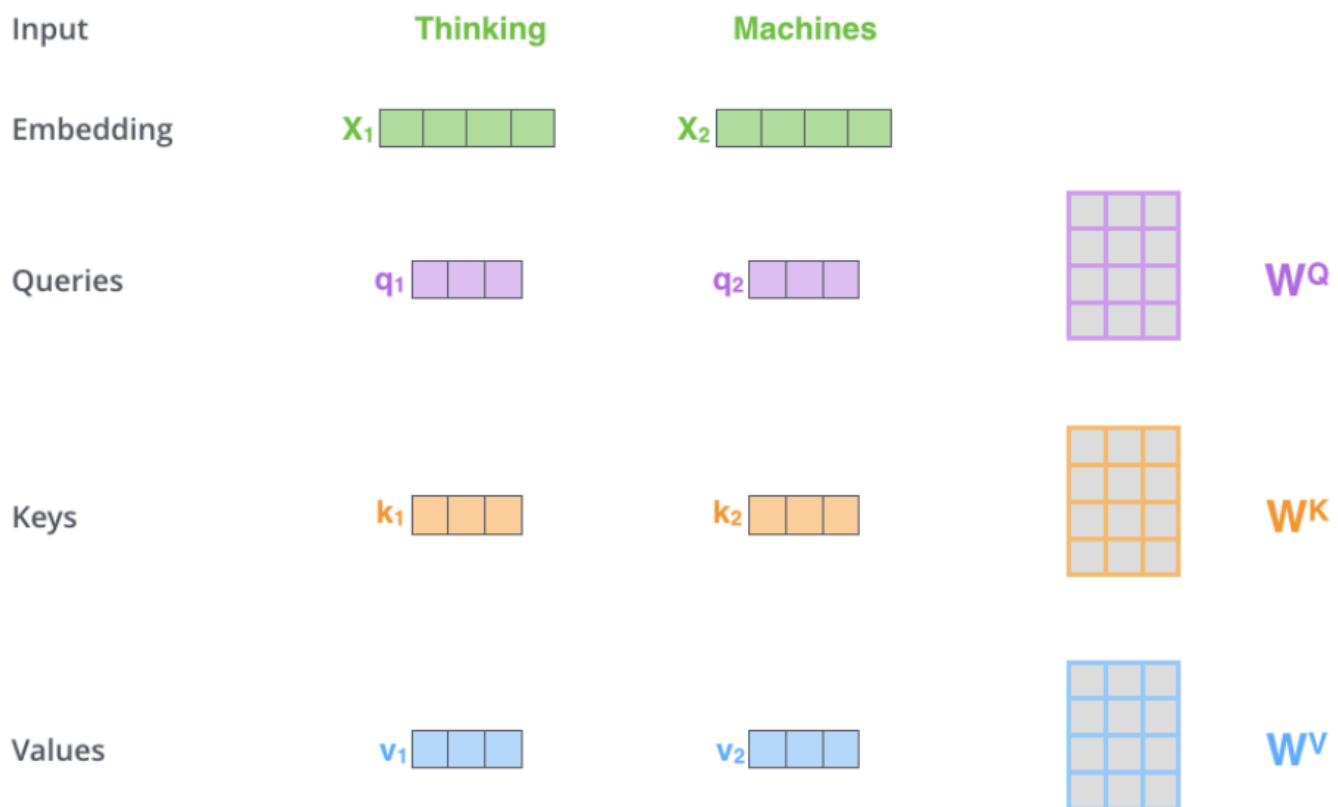
در این بخش، ابتدا نحوه‌ی محاسبه‌ی توجه به خود با استفاده از بردارها را بررسی می‌کنیم و سپس خواهیم دید که این محاسبات چگونه در قالب ماتریس‌ها پیاده‌سازی می‌شوند تا سرعت پردازش آن افزایش یابد.

مرحله‌ی اول: ایجاد بردارهای کوئری (query)، کلید (key) و مقدار (value)

در گام نخست، برای هر کلمه در ورودی انکدر، سه بردار مجزا ایجاد می‌کنیم که عبارت‌اند از:

- بردار کوئری (Query Vector)
- بردار کلید (Key Vector)
- بردار مقدار (Value Vector)

این بردارها از طریق ضرب بردار امبدینگ (embedding) هر کلمه در سه ماتریس متفاوت که مقدار آن‌ها در طی فرایند آموزش مدل به دست می‌آیند، محاسبه می‌شوند.



ایجاد بردارهای کوئری (query)، کلید (key) و مقدار (Value) برای هر کلمه در ورودی انکدر

نکته‌ی مهم:

این بردارهای جدید معمولاً ابعادی کوچک‌تر از بردار امبدینگ اصلی دارند.

به طور مثال، ابعاد بردارهای کوئری، کی و ولیو برابر با ۶۴ است در حالی که ابعاد بردار امبدینگ اصلی ۵۱۲ می‌باشد.

چرا این بردارها کوچک‌تر انتخاب می‌شوند؟

این یک انتخاب معماری است که باعث می‌شود پیاده‌سازی توجه چندسری (Multi-Head Attention) کارآمدتر و کم‌هزینه‌تر شود.

این سه بردار (query, key, value)، مفاهیمی انتزاعی هستند که برای محاسبه‌ی مکانیزم توجه استفاده می‌شوند.

در ادامه با نحوه‌ی محاسبه‌ی توجه به خود (Self-Attention) آشنا شده و نقش هر یک از این بردارها را به‌طور کامل درک خواهید کرد.

مرحله‌ی دوم: محاسبه‌ی امتیاز (score) توجه

پس از تولید بردارهای کوئری (Q)، کلید (K) و مقدار (V)،

باید میزان اهمیت یا ارتباط هر کلمه (یا توکن) را نسبت به سایر کلمات (توکن‌ها) در جمله‌ی ورودی بسنجیم.

برای درک بهتر این موضوع، یک مثال در نظر بگیرید:

فرض کنید که در حال پردازش توجه به خود برای کلمه‌ی "Thinking" هستیم. در این مرحله، باید هر کلمه‌ی موجود در جمله‌ی ورودی را نسبت به این کلمه امتیازدهی کنیم.

امتیاز چگونه محاسبه می‌شود؟

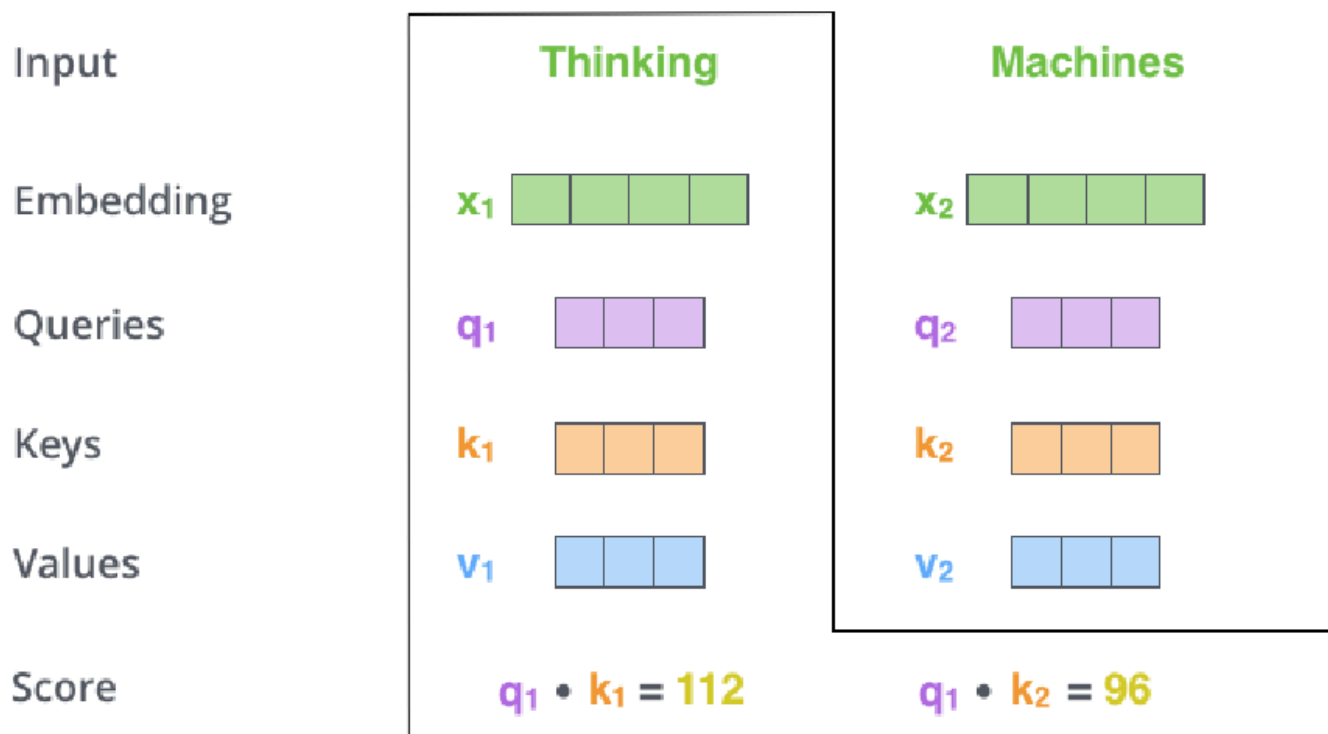
این مقدار از طریق محاسبه‌ی ضرب داخلی (Dot Product) بین بردار کوئری کلمه‌ی موردنظر و بردار کلید سایر کلمات به‌دست می‌آید.

به‌طور مثال، اگر در حال پردازش کلمه‌ی اول باشیم، امتیازات به این شکل محاسبه می‌شوند:

امتیاز اول: حاصل ضرب داخلی بین q_1^* و k_1^{**}

امتیاز دوم: حاصل ضرب داخلی بین q_1^* و k_2^{**}

...



این امتیازات نشان می‌دهند که هر کلمه تا چه حد باید در تعیین معنای کلمه‌ی فعلی مشارکت داشته باشد.

مرحله‌ی سوم و چهارم: نرمال‌سازی امتیازات توجه

پس از محاسبه‌ی امتیازات توجه در مرحله‌ی قبل، باید این مقادیر را نرمال‌سازی کنیم

تا مدل بتواند پردازش بهتری روی اطلاعات ورودی انجام دهد. این کار در دو گام انجام می‌شود:

1. تقسیم امتیازات بر یک مقدار ثابت:

امتیازات محاسبه‌شده بر مقدار قدر مطلق ابعاد کلید یعنی $\sqrt{d_k}$ تقسیم می‌شوند که در اینجا مقدار $\sqrt{64} = 8$ است.

این کار باعث پایدارتر شدن گرادیان‌ها و بهبود عملکرد مدل در طول یادگیری می‌شود.

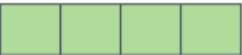
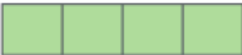
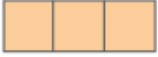
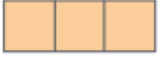
می‌توان از مقادیر دیگری نیز استفاده کرد، اما مقدار $\sqrt{d_k}$ یک انتخاب رایج و پیش‌فرض در معماری ترانسفورمر است.

2. اعمال تابع Softmax:

پس از تقسیم، این امتیازات وارد تابع Softmax می‌شوند.

این تابع باعث می‌شود که تمام امتیازات:

- مثبت شوند (همه مقادیر خروجی بین ۰ و ۱ خواهند بود).
- جمع آن‌ها برابر ۱ شود (خاصیت توزیع احتمالاتی (Probability Distribution)).

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

اکنون این امتیازات نرمال شده نشان می‌دهند که هر کلمه تا چه حد باید در تعیین معنای کلمه‌ی فعلی نقش داشته باشد.

مرحله‌ی پنجم و ششم: وزن‌دهی و ترکیب بردارهای مقدار

در این بخش، از امتیازات نرمال شده‌ی Softmax برای ترکیب اطلاعات مربوط به کلمات مختلف استفاده می‌شود.

این فرآیند در دو مرحله انجام می‌شود:

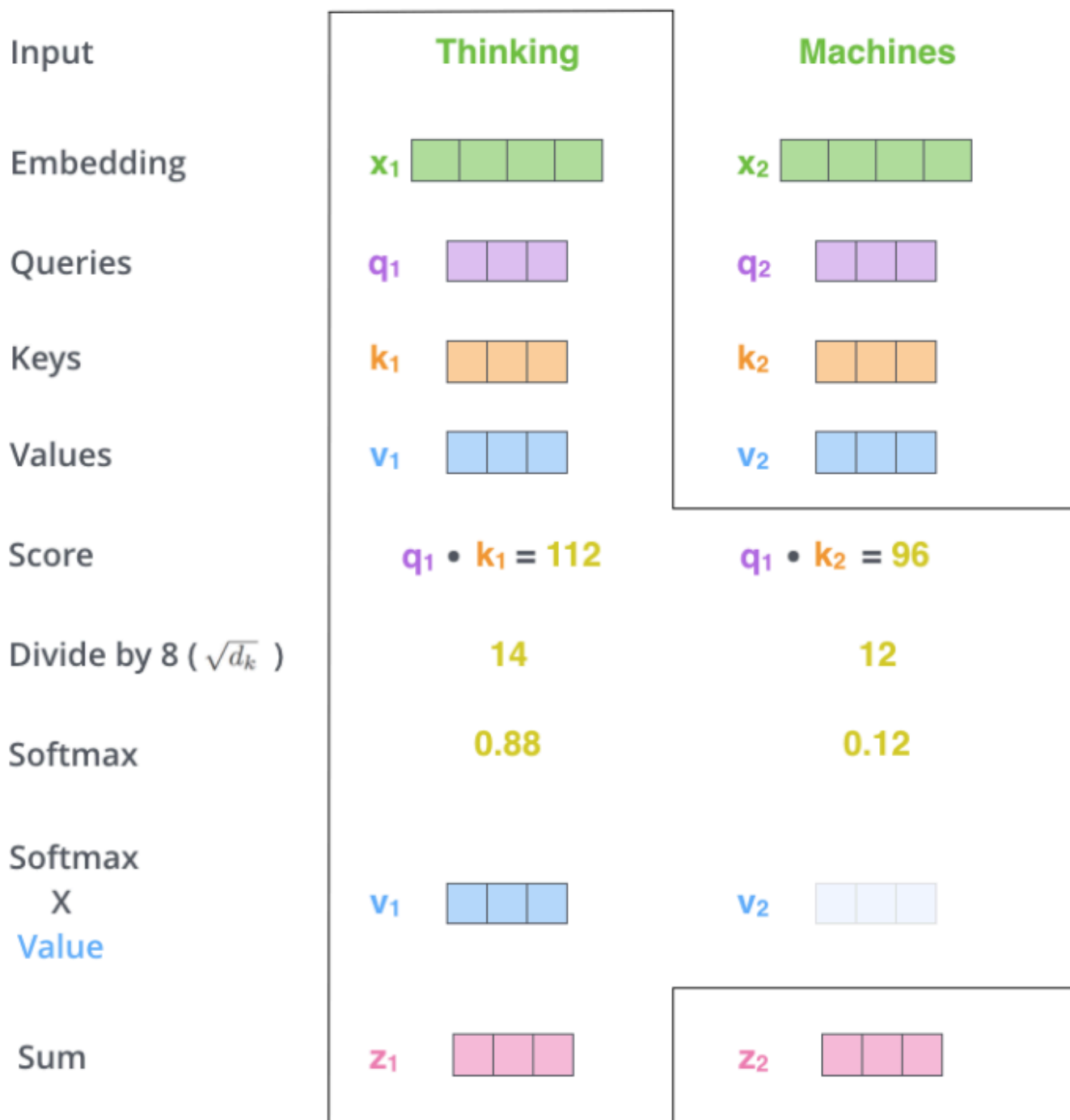
1. وزن‌دهی بردارهای مقدار:

هر بردار مقدار (Value Vector) مربوط به کلمات ورودی، در امتیاز نرمال شده‌ی Softmax مربوط به همان کلمه ضرب می‌شود.

- این کار باعث می‌شود مقادیر مهم، دست‌نخورده باقی بمانند.
- در مقابل، کلمات کم‌اهمیت تقریباً به صفر میل کنند (زیرا در اعداد بسیار کوچک، مثل 0.001، ضرب می‌شوند).

2. ترکیب بردارهای وزن‌دهی شده:

- در این مرحله، تمام بردارهای مقدار وزن دار با یکدیگر جمع می‌شوند.
- خروجی نهایی این مرحله برداری جدید برای موقعیت فعلی در دنباله‌ی ورودی است.
- این بردار شامل اطلاعاتی از کلمات قبلی که بیشترین تأثیر را دارند، خواهد بود.



نتیجه‌ی نهایی این فرآیند بردار خروجی لایه‌ی توجه برای موقعیت موردنظر است.

این بردار اکنون حاوی اطلاعات مفیدی از کلمات مرتبط با کلمه‌ی فعلی می‌باشد.

در این مرحله، محاسبات مربوط به توجه به خود (Self-Attention) به پایان می‌رسد.

برداری که در خروجی این فرآیند تولید می‌شود، حاوی اطلاعات پردازش شده از تمامی کلمات قبلی مرتبط با موقعیت فعلی است.

این بردار اکنون می‌تواند به‌عنوان ورودی شبکه‌ی عصبی کاملاً متصل (Feed-Forward Neural Network) مورد استفاده قرار گیرد.

بهینه‌سازی محاسبات توجه به خود (self attention)

تاکنون فرآیند توجه را در سطح بردارهای مجزا بررسی کردیم، در عمل پیاده‌سازی واقعی، این محاسبات با استفاده از ماتریس‌ها انجام می‌شود تا کارایی پردازشی افزایش یابد.

- استفاده از ماتریس‌ها به‌جای بردارهای منفرد، امکان پردازش موازی را فراهم می‌آورد.
- این تکنیک منجر به افزایش سرعت محاسبات در مدل‌های زبانی مدرن می‌شود.

در بخش بعد، به نحوه‌ی فرموله‌سازی ریاضی و پیاده‌سازی ماتریسی توجه به خود خواهیم پرداخت.

محاسبات ماتریسی مکانیزم توجه به خود

اولین گام در پیاده‌سازی توجه به خود در قالب ماتریسی، محاسبه‌ی ماتریس‌های پرس‌وجو (Query)، کلید (Key) و مقدار (Value) است.

برای انجام این محاسبات، ابتدا بردارهای تعبیه شده‌ی ورودی (embedding) را در یک ماتریس X بسته‌بندی کرده و سپس آن را در ماتریس‌های وزن W_Q ، W_K و W_V که در طول فرآیند آموزش یاد گرفته شده‌اند، ضرب می‌کنیم:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

در این رابطه:

- * X (ابعاد: $n \times d$) ماتریس بردارهای امبدینگ‌شده‌ی ورودی
- * W_Q, W_K, W_V (ابعاد: $d \times d_k$) ماتریس‌های وزن آموزش‌دیده‌شده
- * Q, K, V (ابعاد: $n \times d_k$) ماتریس‌های پرس‌وجو، کلید و مقدار

این تبدیل، ورودی‌های اصلی را به فضای توجه نگاشت می‌کند و امکان محاسبه‌ی امتیازات توجه را فراهم می‌آورد.

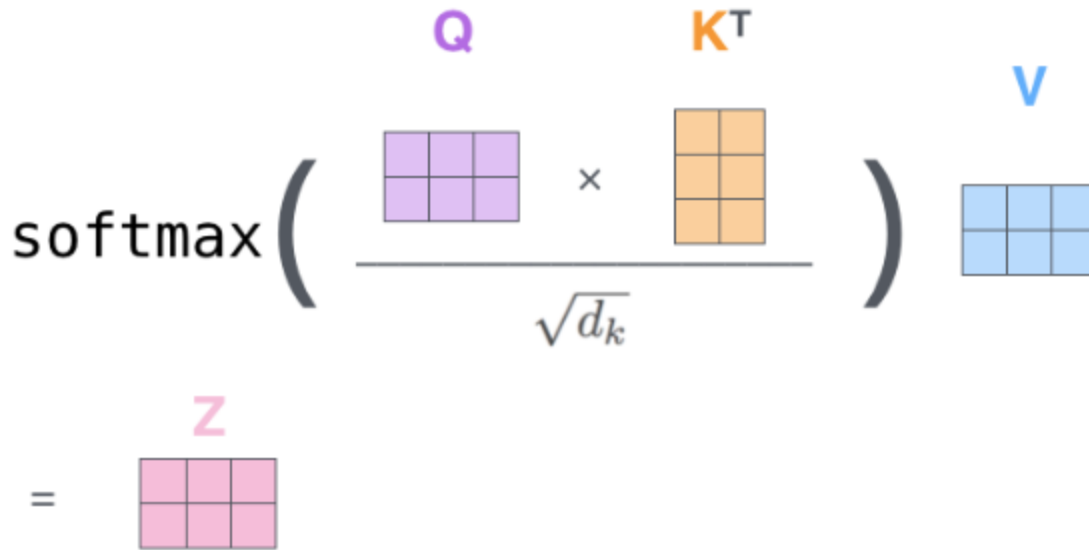


در گام بعدی، از این ماتریس‌ها برای محاسبه‌ی امتیازات توجه و نرمال‌سازی آن‌ها استفاده خواهیم کرد.

فرمول نهایی محاسبات ماتریسی مکانیزم توجه به خود

از آنجایی که در محاسبات مکانیزم توجه با ماتریس‌ها سر و کار داریم، می‌توانیم گام‌های دوم تا ششم را در یک رابطه‌ی واحد خلاصه کنیم تا خروجی لایه‌ی توجه به خود محاسبه شود:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



در این رابطه:

- * **Q** ماتریس پرس و جو (ابعاد: $n \times d_k$)
- * **K** ماتریس کلید (ابعاد: $n \times d_k$)
- * **V** ماتریس مقدار (ابعاد: $n \times d_k$)
- * d_k Attention is All You Need معمولاً مقدار آن 64 در مقاله‌ی اصلی) بُعد فضای توجه

• عملگر softmax امتیازات توجه را نرمال‌سازی کرده تا مقادیر به صورت احتمالی تفسیر شوند.

این فرمول نهایی محاسبات توجه به خود (self attention) است که هسته‌ی اصلی عملکرد مدل‌های ترنسفورمر (Transformer) را تشکیل می‌دهد.

توجه چندسری (multi head attention): هیولای چندسری!

مقاله‌ی Attention is All You Need مکانیزم توجه به خود را با معرفی "توجه چندسری" بهبود بخشید. این تکنیک باعث افزایش عملکرد لایه‌ی توجه از دو طریق می‌شود:

1. افزایش توانایی مدل در تمرکز روی موقعیت‌های مختلف:

در مثال قبلی، مقدار خروجی z_1 شامل اطلاعاتی از سایر رمزگذاری‌ها است، اما ممکن است تحت تأثیر خود کلمه‌ی اصلی قرار بگیرد. به عنوان مثال، در جمله‌ی:

The animal didn't cross the street because it was too tired

مدل باید متوجه شود که "it" به "animal" اشاره دارد، نه "street". توجه چندسری کمک می‌کند که مدل بتواند وابستگی‌های متعددی را به طور همزمان تشخیص دهد.

2. ایجاد زیرفضاهای نمایشی مختلف:

در مکانیزم توجه چندسری (multi head attention)، به جای داشتن تنها یک مجموعه از ماتریس‌های وزن $\{Q, K, V\}$ ، ما چندین مجموعه داریم. به عنوان مثال، مدل Transformer معمولاً از 8 سری توجه (Attention Heads) استفاده می‌کند، به این ترتیب:

- هر سری دارای مجموعه‌ای مستقل از ماتریس‌های $\{W_Q, W_K, W_V\}$ است.
- این ماتریس‌ها ابتدا به صورت تصادفی مقداردهی اولیه می‌شوند.
- در طی فرآیند آموزش، هر سری یاد می‌گیرد که ورودی‌ها را به یک زیرفضای نمایشی متفاوت بفرستد.

این تکنیک به مدل اجازه می‌دهد که به ابعاد معنایی مختلفی از جمله وابستگی‌های دستوری، معنایی، و ارتباط بین کلمات در متن توجه کند.

در مکانیزم توجه چندسری، در واقع برای هر سری توجه (attention Head)، سه ماتریس وزن مستقل برای بردارهای پرسش (Q)، کلید (K)، و مقدار (V) در نظر گرفته می‌شود. این بدان معناست که برخلاف توجه تک‌سری که تنها یک مجموعه از ماتریس‌های $\{W_Q, W_K, W_V\}$ داشت، در توجه چندسری چندین مجموعه مستقل از این ماتریس‌ها داریم.

- هر سری توجه (attention Head)، مجموعه‌ای از ماتریس‌های وزن مجزا دارد:

$$W_Q, W_K, W_V \text{ یارب } H, 1 =$$

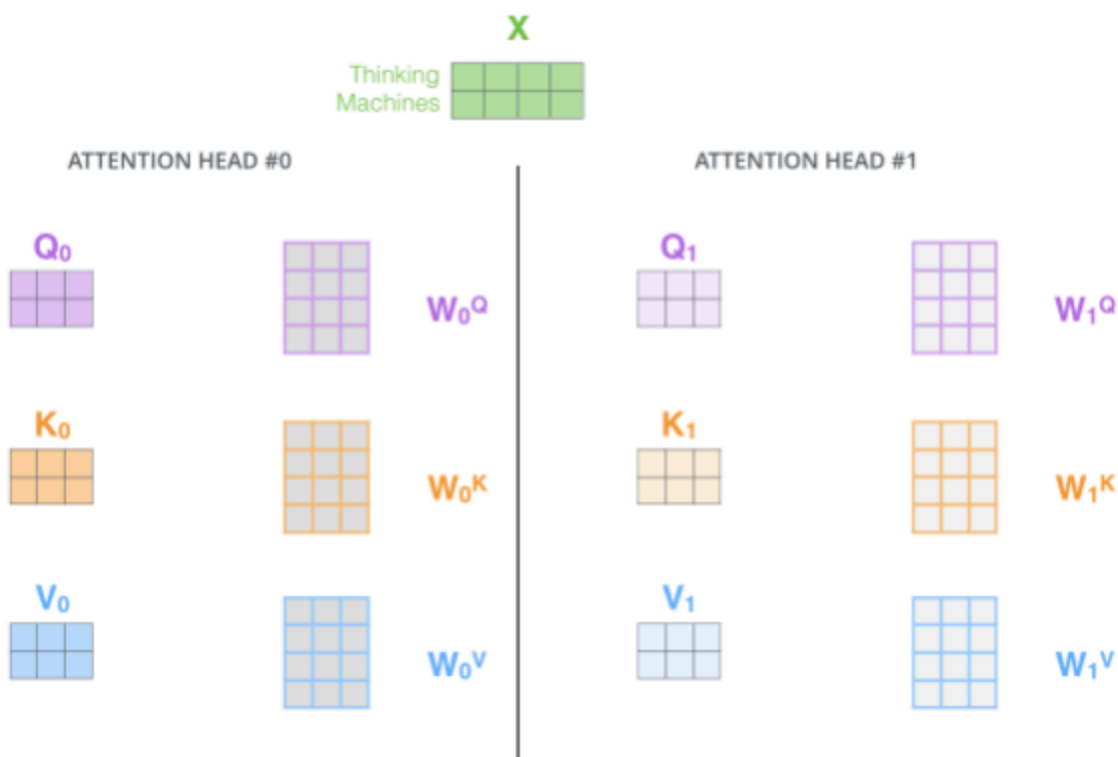
که در آن H تعداد کل سری‌های توجه است.

- همانند توجه تک‌سری، ابتدا ماتریس ورودی X را در این ماتریس‌های وزن ضرب می‌کنیم تا ماتریس‌های Q, K, V متناظر با هر سری تولید شوند:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

در نهایت، تمام خروجی‌های به دست آمده از همه‌ی سری‌های توجه را به هم می‌چسبانیم و آن‌ها را در یک ماتریس وزن اضافی W_O ضرب می‌کنیم تا خروجی نهایی تولید شود:

$$Z = \text{Concat}(Z^1, Z^2, \dots, Z^H)W_O$$



محاسبه توجه چندسری با استفاده از ماتریس‌های وزن مستقل

همان‌طور که در بخش‌های قبل اشاره شد، در مکانیزم توجه چندسری (Multi-Head Attention)، به جای یک مجموعه‌ی واحد از ماتریس‌های وزن W_Q , W_K , W_V ، چندین مجموعه مستقل از این ماتریس‌ها وجود دارد. این باعث می‌شود مدل بتواند به جنبه‌های مختلفی از توزیع توجه توجه کند و الگوهای پیچیده‌تری را در داده‌ها یاد بگیرد.

- اگر H سری توجه داشته باشیم، در هر سری، عملیات محاسباتی توجه مانند آنچه در توجه تک‌سری بیان شد، انجام می‌شود اما به‌صورت موازی برای هر سری.
- برای هر سری $h = 1, 2, \dots, H$ ، سه ماتریس وزن مستقل تعریف می‌شود:

$$W_Q, W_K, W_V$$

و عملیات ضرب ماتریسی برای محاسبه‌ی بردارهای پرسش (Q^h) ، کلید (K^h) ، و مقدار (V^h) انجام می‌شود:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- سپس محاسبه‌ی توجه برای هر سری به‌صورت زیر انجام می‌شود:

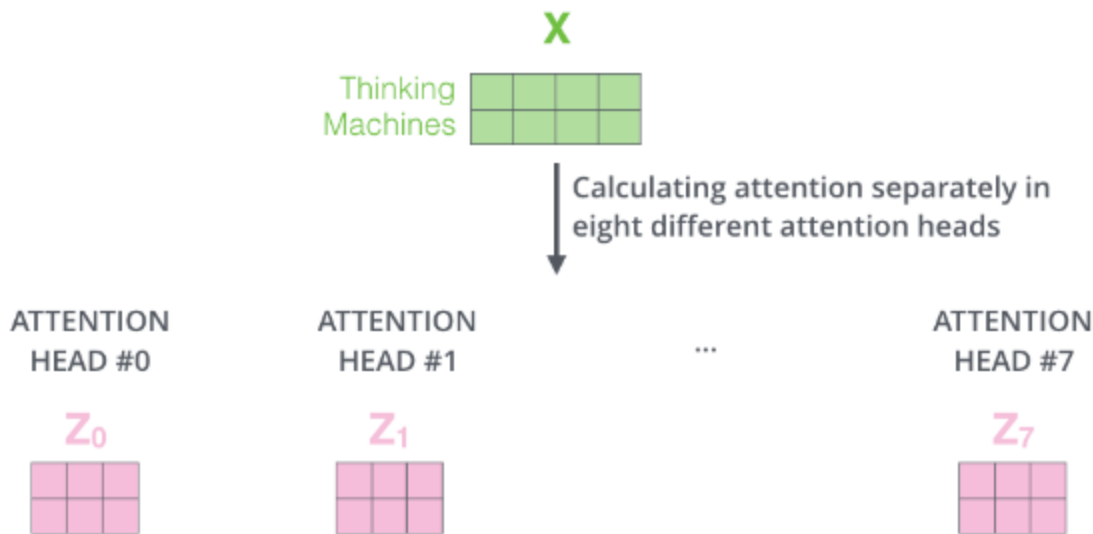
$$Z = \text{softmax} \left(\frac{Q(K)^T}{\sqrt{d_k}} \right) V$$

که در آن d_k بعد بردارهای کلید است.

- در نهایت، تمام خروجی‌های حاصل از سری‌های توجه به‌هم چسبانده می‌شوند:

$$Z = \text{Concat}(Z^1, Z^2, \dots, Z^H)W_O$$

که در آن W_0 ماتریس وزن نهایی برای ترکیب خروجی‌های سری‌های توجه است.



ادغام خروجی‌های چندسری توجه و ارسال به شبکه‌ی عصبی کاملاً متصل

پس از انجام عملیات توجه چندسری (Multi-Head Attention)، خروجی‌های به‌دست‌آمده از هر سری توجه، به‌صورت مجموعه‌ای از ماتریس‌های مستقل $Z^1, Z^2, \dots, Z^H$ در اختیار مدل قرار می‌گیرند. با این حال، لایه‌ی بعدی مدل که یک شبکه‌ی عصبی کاملاً متصل است، انتظار دارد که یک ماتریس منفرد به آن ارسال شود.

برای حل این چالش، دو مرحله‌ی اصلی برای تجميع اطلاعات انجام می‌شود:

1. اتصال ماتریس‌های خروجی توجه چندسری

ابتدا، تمام بردارهای خروجی Z^h از سری‌های مختلف، در امتداد بعد ویژگی‌ها (Feature Dimension) به‌هم چسبانده می‌شوند تا یک ماتریس واحد تشکیل شود:

$$Z = \text{Concat}(Z^1, Z^2, \dots, Z^H)$$

در اینجا، Z دارای بعدی برابر با d_{model} است، که معمولاً برابر با 512 یا 1024 در مدل‌های مدرن است.

2. ضرب در ماتریس وزن خروجی W_0

سپس، این ماتریس در یک ماتریس وزن خروجی W_0 ضرب می‌شود تا ویژگی‌های ترکیبی از تمام سری‌های توجه استخراج شده و به شبکه‌ی عصبی کاملاً متصل ارسال شوند:

$$Z_{\text{final}} = ZW_0$$

در اینجا، W_0 ماتریسی با ابعاد $(H * d_k, d_{\text{model}})$ است که برای کاهش ابعاد به مقدار مورد نیاز، تنظیم می‌شود.

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

x



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



تجميع تمامی اجزای توجه چندسری

توجه چندسری (Multi-Head Self-Attention) از چندین عملیات ماتریسی تشکیل شده است که با هم ترکیب می‌شوند تا اطلاعات متنوعی را از توکن‌های مختلف ورودی استخراج کنند. در این بخش، تمامی این محاسبات را در یک نگاه کلی ارائه می‌دهیم تا درک جامعی از فرآیند ایجاد شود.

1. ابتدا، توکن‌های ورودی به بردارهای امبدینگ‌شده (Embeddings) تبدیل شده و در قالب ماتریس X نمایش داده می‌شوند.
2. سپس، بردارهای ورودی در ماتریس‌های وزن مخصوص سری‌های توجه ضرب می‌شوند تا سه ماتریس کلیدی تولید شود:

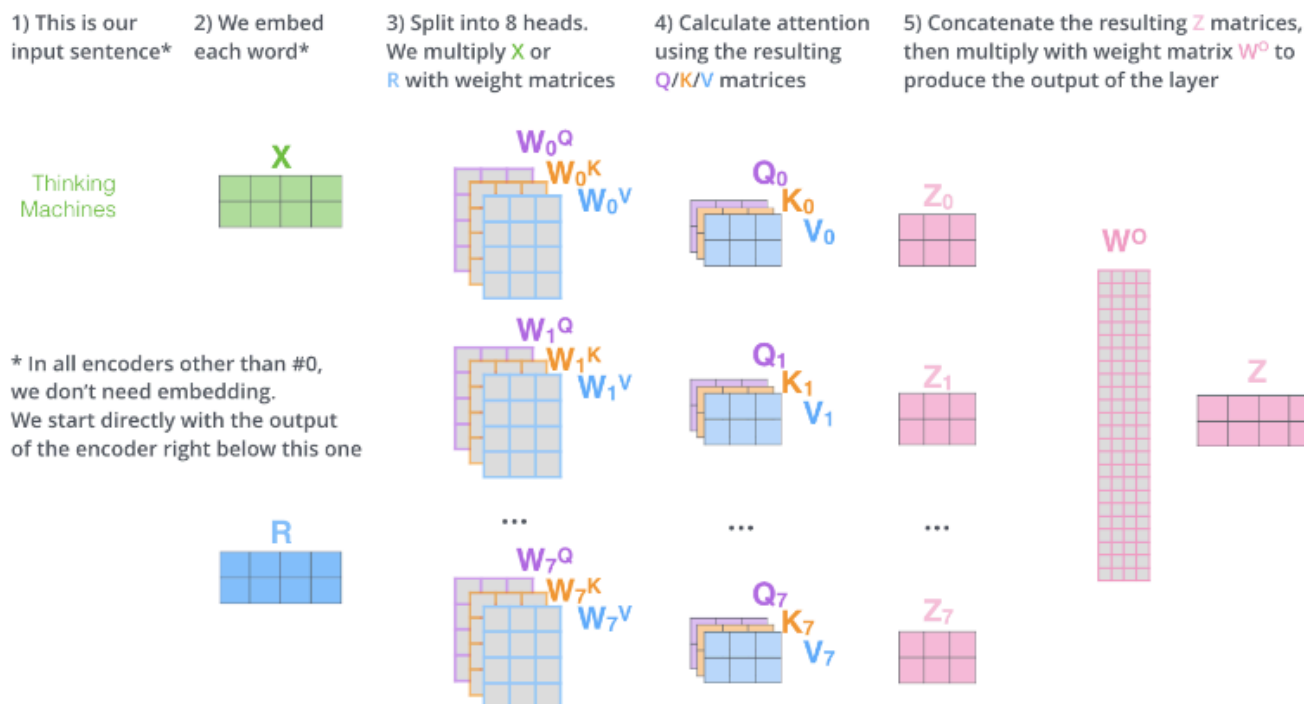
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. محاسبه‌ی امتیاز توجه از طریق ضرب نقطه‌ای بردارهای پرسش (Q) و کلید (K) و اعمال تابع نرم‌سازی Softmax:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

4. این فرآیند برای هر سری توجه (H) به صورت مستقل اجرا می‌شود.
5. خروجی‌های سری‌های مختلف توجه Z^h در یک ماتریس نهایی ادغام شده و سپس در ماتریس وزن خروجی W_O ضرب می‌شود:

$$Z_{\text{final}} = \text{Concat}(Z^1, Z^2, \dots, Z^H) W_O$$



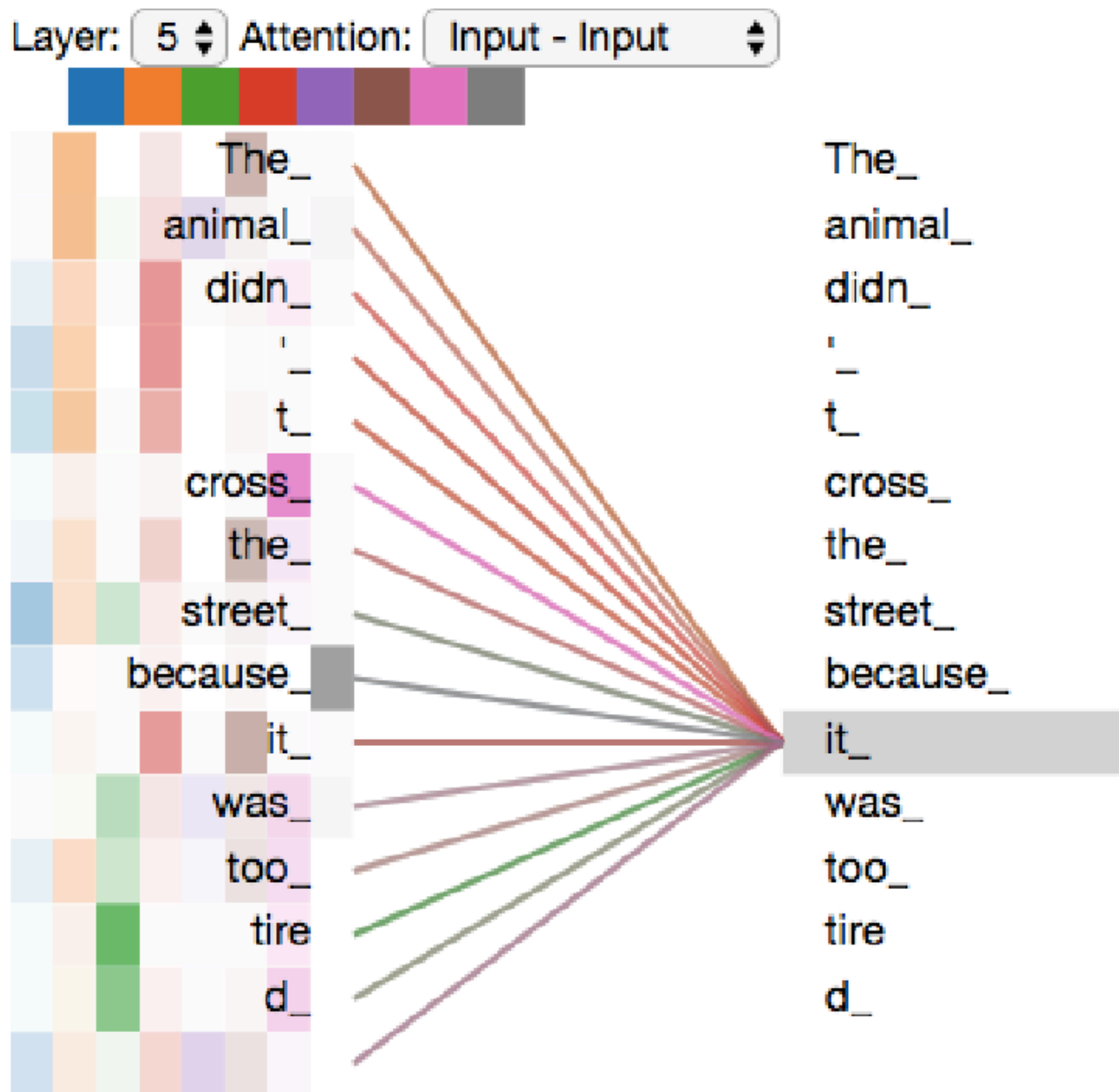
این مکانیزم به مدل اجازه می‌دهد که در چندین فضای نمایش مختلف به‌طور همزمان پردازش انجام دهد و وابستگی‌های پیچیده را بهتر درک کند. در بخش‌های بعدی، به نحوه‌ی استفاده از خروجی این لایه در شبکه‌ی عصبی کاملاً متصل (Feed-Forward Neural Network) خواهیم پرداخت.

تمرکز سری‌های توجه بر روی کلمه‌ی "it"

اکنون که سری‌های توجه (Attention Heads) را بررسی کردیم، بیایید مثال قبلی را مرور کنیم تا ببینیم هر سری توجه بر چه بخشی از جمله تمرکز دارد.

- در هنگام پردازش کلمه‌ی "it"، یکی از سری‌های توجه بیشترین تمرکز را بر روی کلمه‌ی "the animal" دارد.
- سری دیگری از توجه بر روی کلمه‌ی "tired" متمرکز شده است.

این بدین معنی است که نمایش مدل از کلمه‌ی "it" اطلاعاتی را از هر دو کلمه‌ی "animal" و "tired" در خود جای داده است. به‌عبارتی دیگر، مدل توانسته ارتباط معنایی بین کلمات را استخراج کرده و در نمایش برداری آن‌ها لحاظ کند.



ادغام تمامی سَری‌های توجه

زمانی که تمامی سَری‌های توجه را در یک تصویر لحاظ کنیم، تحلیل و تفسیر آن‌ها دشوارتر می‌شود. این امر به این دلیل است که هر سَری توجه در حال پردازش نوع خاصی از روابط بین توکن‌ها است. برخی از این روابط ممکن است برای درک ساختار جمله و برخی دیگر برای درک روابط معنایی بین واژگان مورد استفاده قرار گیرند.

با این وجود، این چندگانگی در تمرکز توجه همان چیزی است که باعث قدرت فوق‌العاده مدل‌های مبتنی بر Transformer در درک و پردازش زبان طبیعی می‌شود.

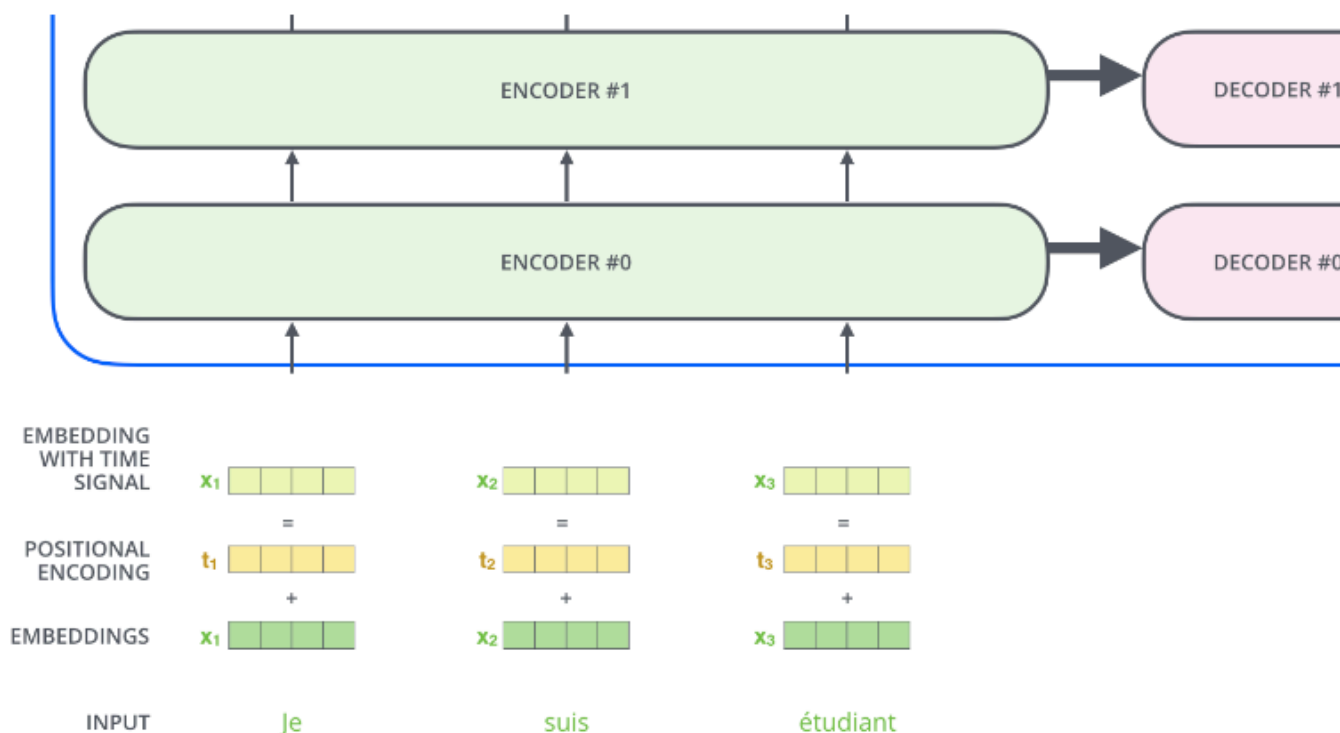
نمایش ترتیب کلمات در دنباله با رمزگذاری موقعیتی

در مدل‌های Transformer، برخلاف مدل‌های بازگشتی (RNN)، هیچ وابستگی ذاتی بین توکن‌های ورودی وجود ندارد. این بدان معناست که مدل باید ترتیب کلمات را به نحوی در نظر بگیرد تا بتواند پردازش زبان طبیعی را به درستی انجام دهد.

به همین دلیل، یک بردار رمزگذاری موقعیتی (Positional Encoding) به هر امبدینگ ورودی (Embedding) اضافه می‌شود. این بردارها از یک الگوی خاص پیروی می‌کنند که مدل می‌تواند با استفاده از آن موقعیت هر کلمه و فاصله‌ی آن را از دیگر کلمات در دنباله درک کند.

ساختار رمزگذاری موقعیتی

- این بردارها به گونه‌ای طراحی شده‌اند که فاصله‌های معنایی بین بردارهای امبدینگ شده را معنادار کنند.
- زمانی که بردارهای امبدینگ در فضای Q/K/V نگاشت می‌شوند و در محاسبات توجه (attention) و در عملیات ضرب نقطه‌ای (Dot-Product Attention) مورد استفاده قرار می‌گیرند، ترتیب کلمات حفظ می‌شود.
- رمزگذاری موقعیتی به مدل اجازه می‌دهد تا وابستگی‌های معنایی را بر اساس موقعیت‌های نسبی کلمات در جمله برقرار کند.

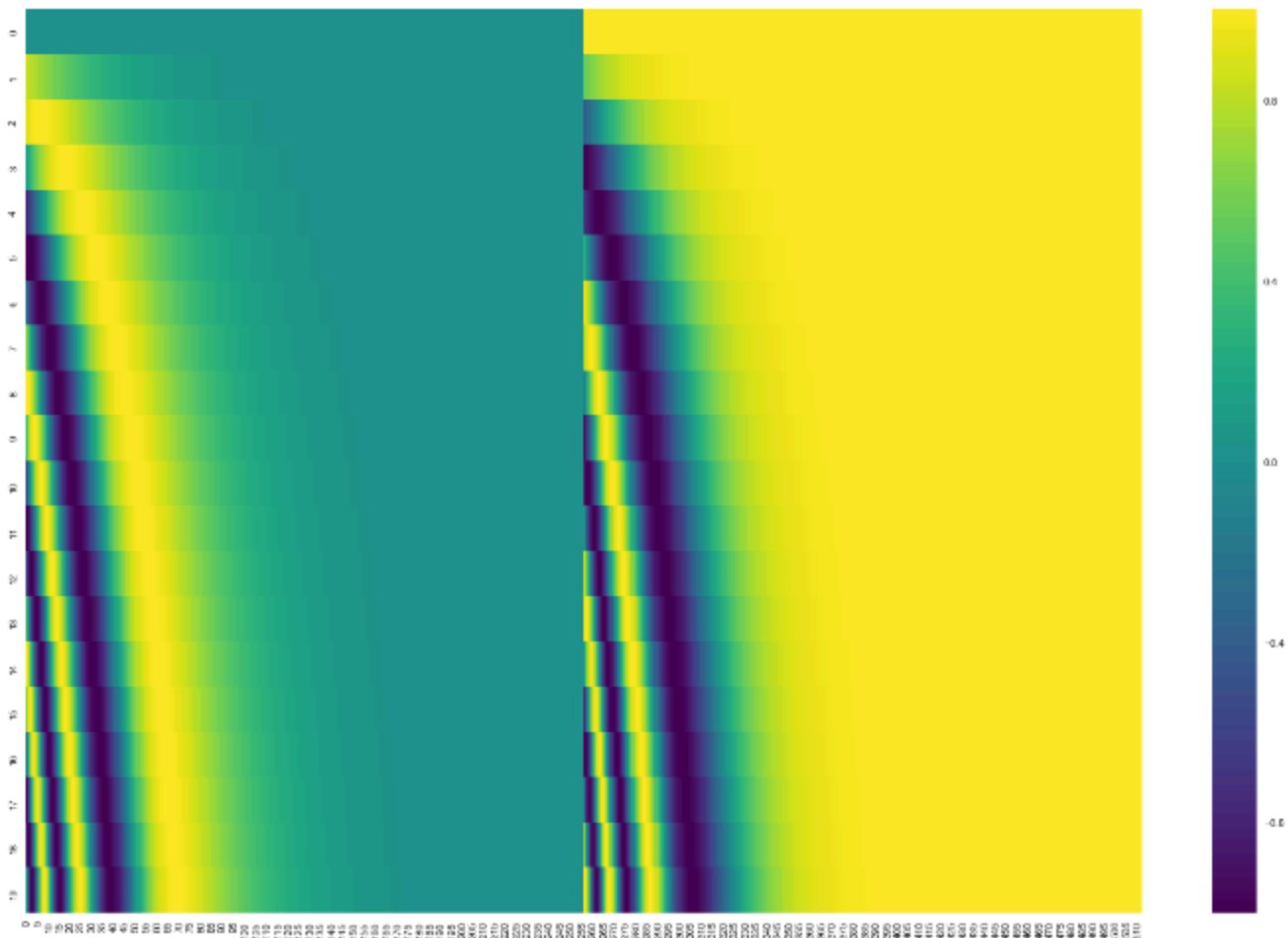


برای درک بهتر، فرض کنید بردار امبدینگ دارای ابعاد ۴ است. در این حالت، بردارهای رمزگذاری موقعیتی به صورت شکل زیر خواهند داشت:



الگوی عددی بردارهای رمزگذاری موقعیتی

در شکل زیر، هر سطر متناظر با یک رمزگذاری موقعیتی برای یک کلمه در یک جمله‌ی ورودی است. هر سطر شامل ۵۱۲ مقدار است که در بازه‌ی ۱ تا ۱- قرار دارند. برای نمایش الگو، مقادیر با رنگ‌بندی خاص نمایش داده شده‌اند.



همان‌طور که مشاهده می‌شود، بردارهای رمزگذاری موقعیتی دارای دو بخش متقارن هستند:

- نیمه‌ی چپ بردارها توسط تابع سینوسی محاسبه شده‌اند.
- نیمه‌ی راست بردارها توسط تابع کسینوسی محاسبه شده‌اند.

فرمول رمزگذاری موقعیتی

فرمول ریاضی رمزگذاری موقعیتی در بخش ۳.۵ مقاله‌ی اصلی معرفی شده است. پیاده‌سازی آن نیز در تابع `get_timing_signal_1d()` ارائه شده است.

ویژگی مهم:

این روش به مدل اجازه می‌دهد که برای جملاتی طولانی‌تر از جملات موجود در مجموعه‌ی آموزشی، به‌درستی تعمیم یابد. به این ترتیب، اگر یک مدل آموزش‌دیده بخواهد جمله‌ای را پردازش کند که از لحاظ طولانی بودن فراتر از داده‌های آموزشی است، همچنان قادر به درک ترتیب کلمات خواهد بود.

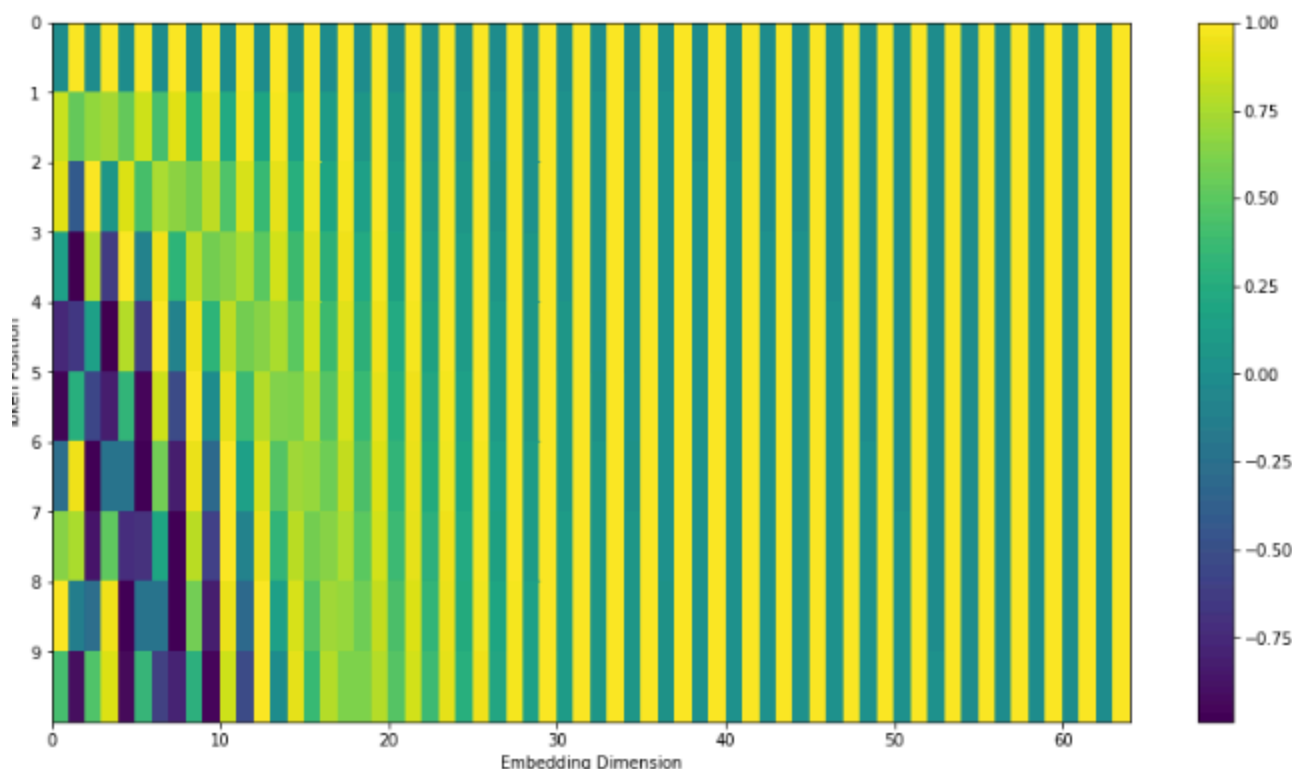
تفاوت روش رمزگذاری موقعیتی در پیاده‌سازی‌های مختلف

رمزگذاری موقعیتی که در بخش قبل مشاهده کردید، مربوط به پیاده‌سازی Tensor2Tensor از مدل Transformer است. با این حال، روش ارائه‌شده در مقاله‌ی اصلی کمی متفاوت است. تفاوت اصلی در این است که:

- در Tensor2Tensor، بردارهای رمزگذاری موقعیتی به صورت مستقیم به هم الحاق (Concatenate) می‌شوند.
- در مقاله‌ی اصلی، این دو سیگنال به جای الحاق شدن، به صورت درهم‌تنیده (Interleaved) ترکیب می‌شوند.

این تفاوت ممکن است بر روی نحوه‌ی پردازش ترتیب کلمات تأثیر بگذارد، اما هر دو روش هدف یکسانی را دنبال می‌کنند.

شکل زیر، نحوه‌ی ترکیب سیگنال‌ها در روش مقاله‌ی اصلی را نشان می‌دهد:



به این ترتیب، در مدل اصلی:

- مقدارهای سینوسی و کسینوسی به صورت متناوب در بردار جای‌گذاری می‌شوند.
- این کار باعث می‌شود که ترتیب موقعیتی کلمات به شکل هموارتر و بدون نیاز به تقسیم واضح بین دو بخش بردار نمایش داده شود.

اتصالات باقی‌مانده (Residual Connections) در معماری Transformer

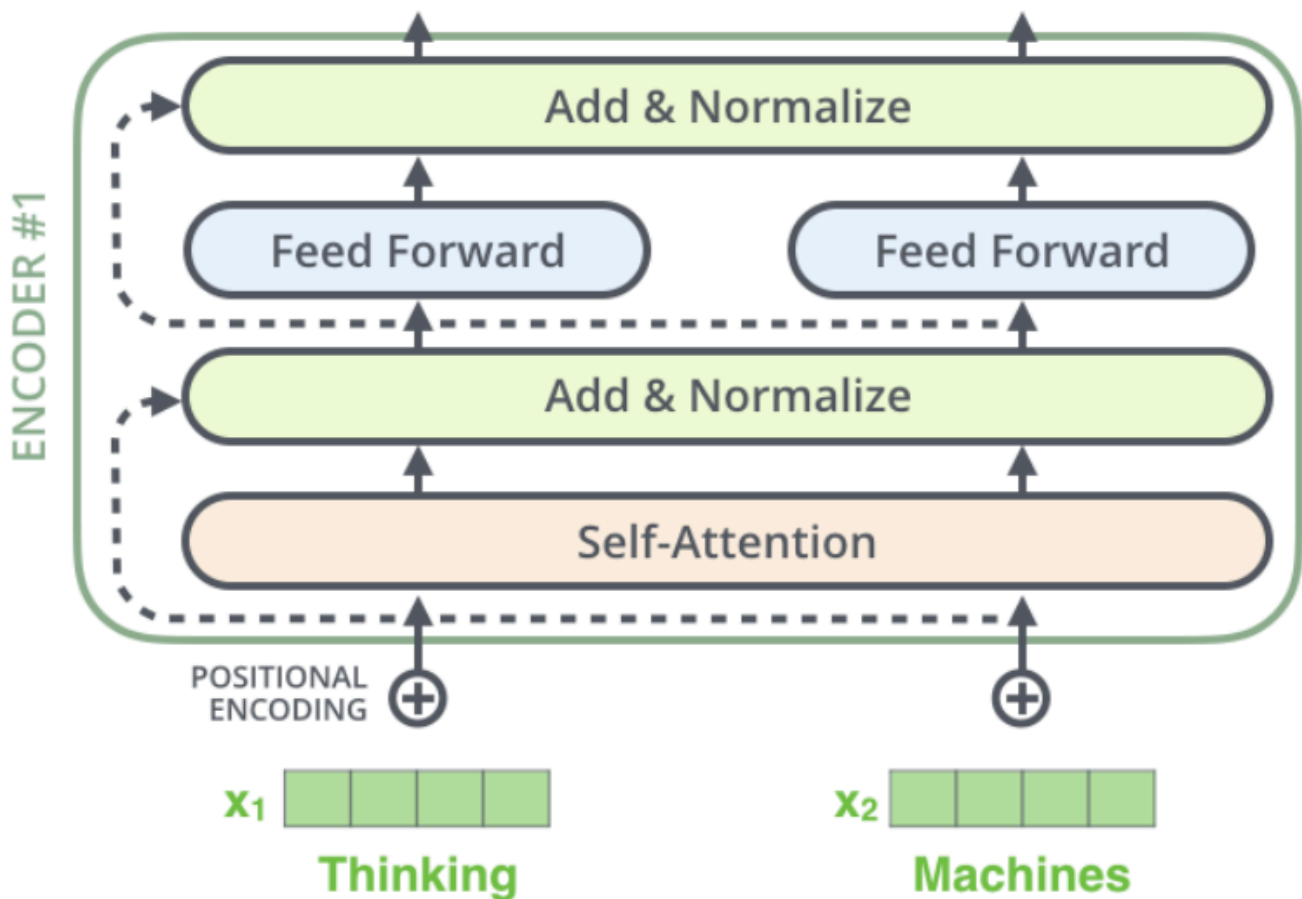
یکی از جزئیات مهم در معماری رمزگذار (Encoder) که قبل از ادامه‌ی بحث باید به آن اشاره کنیم، وجود اتصالات باقی‌مانده (Residual Connections) در هر زیرلایه است. همان طور که قبلاً نیز بیان کردیم در هر رمزگذار و رمزگشا، دو زیرلایه‌ی مهم شامل:

1. لایه‌ی توجه به خود (Self-Attention)

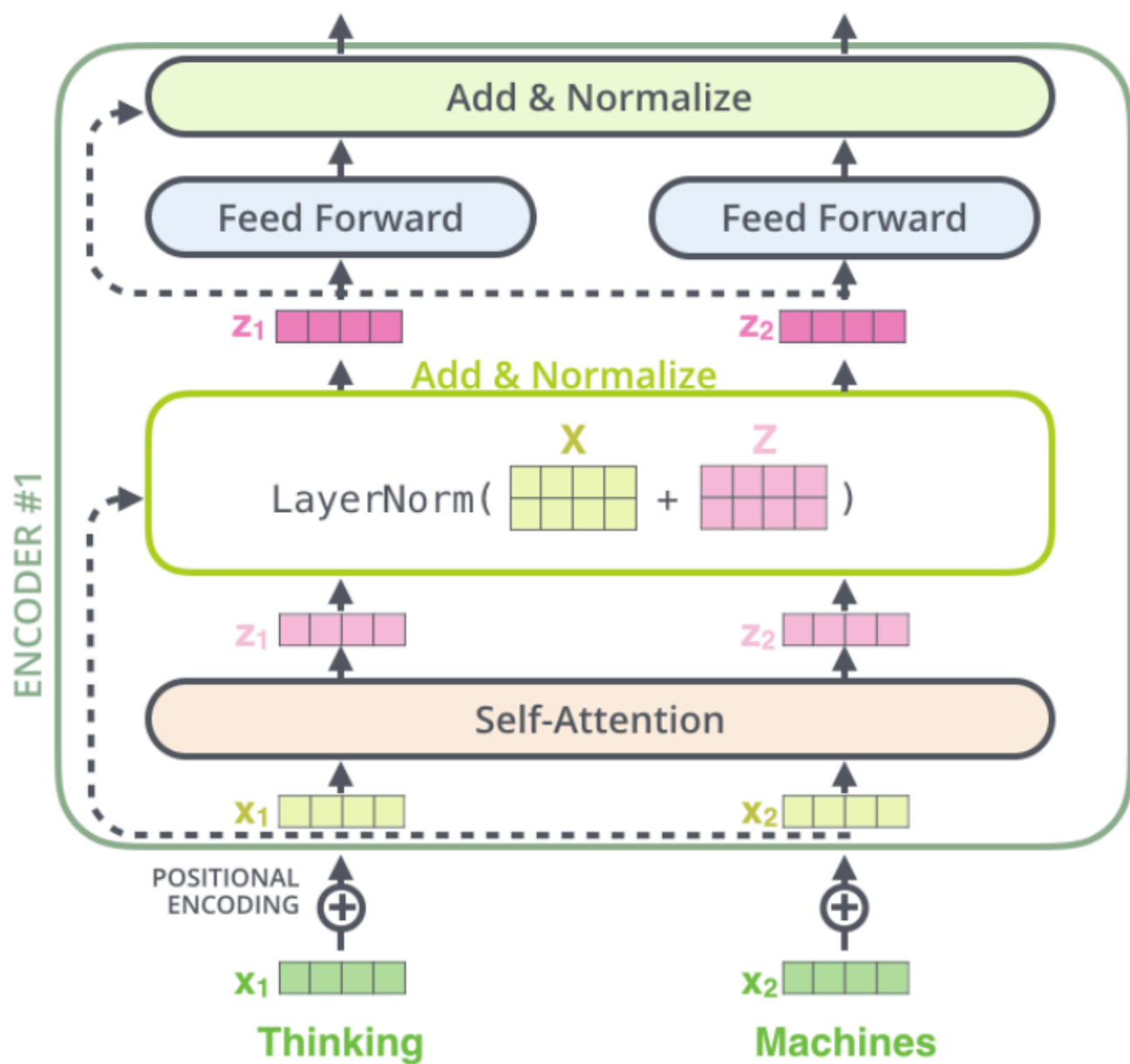
2. شبکه‌ی عصبی کاملاً متصل (Feed-Forward Neural Network, FFNN)

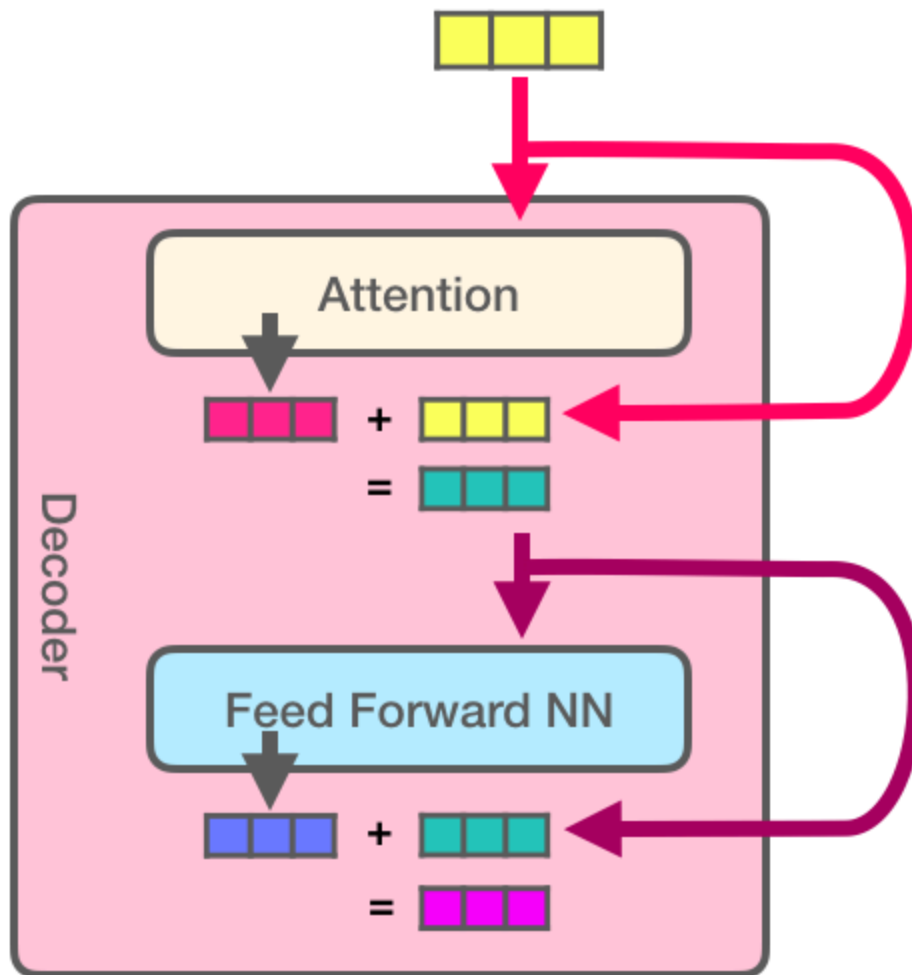
به طور کلی در یادگیری عمیق (Deep Learning)، به‌ویژه در شبکه‌های عصبی مانند ResNet، اتصالات باقی‌مانده یا همان "Residual Connections" به نوعی اتصال گفته می‌شود که ورودی یک لایه را بدون تغییر یا با تغییر اندک مستقیماً به خروجی لایه بعدی اضافه می‌کند. این کار باعث حل مشکل **نابود شدن گرادینت** در شبکه‌های بسیار عمیق می‌شود و آموزش مدل را آسان‌تر می‌کند.

در معماری ترنسفورمر نیز هرکدام از این زیرلایه‌های بلوک رمزگذار و رمزگشا نیز یک اتصال باقی‌مانده دارند که خروجی را با ورودی اولیه (خروجی مرحله قبل) جمع می‌کند. البته این اتصال قبل از عملیات نرمال‌سازی لایه‌ای (Layer Normalization) اعمال می‌شود.



اگر بخواهیم بردارها و عملیات لایه نرمال سازی مرتبط با مکانیزم توجه به خود (self attention) را تجسم کنیم، به شکل زیر خواهد بود:





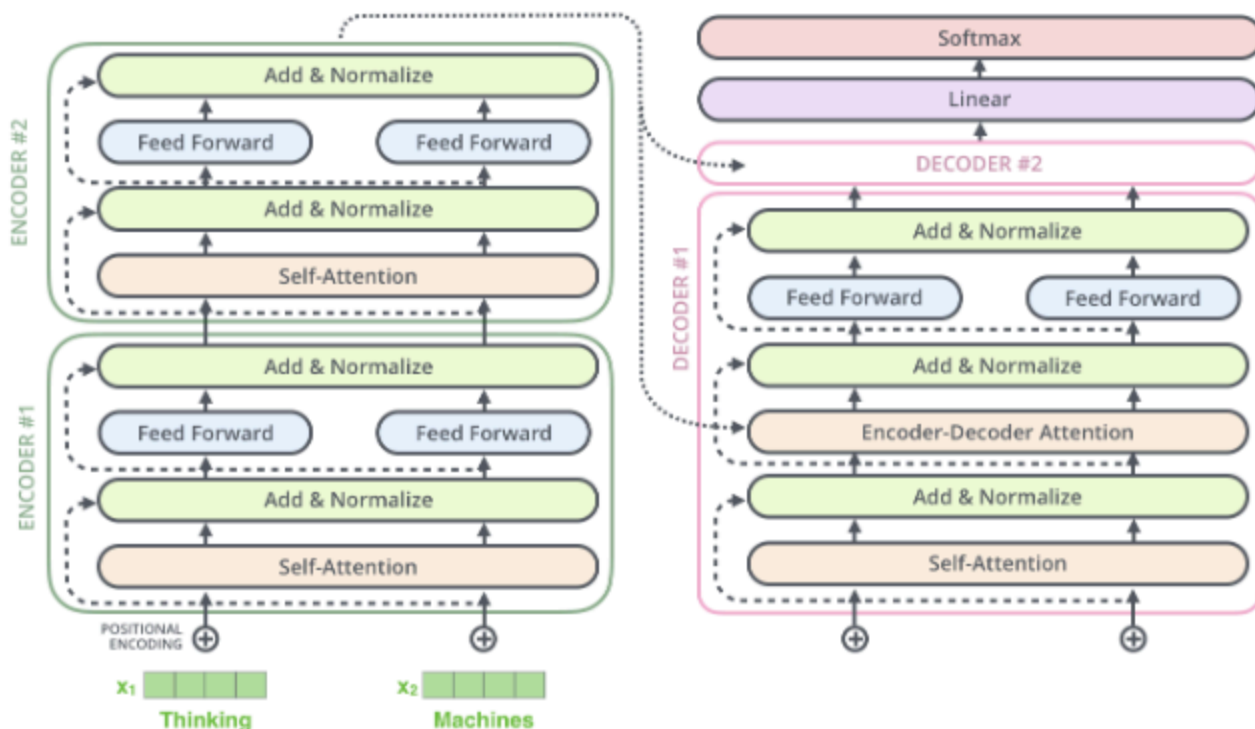
اهمیت اتصالات باقی‌مانده:

به طور کلی افزودن این گونه از اتصالات مزایای زیر را به همراه دارد:

- به بهبود جریان گرادیان‌ها در طول مراحل آموزش کمک می‌کنند.
- مانع از ناپدید شدن گرادیان‌ها می‌شوند که یک چالش اساسی در مدل‌های عمیق است.
- باعث می‌شوند که مدل به اطلاعات اصلی دسترسی مستقیم داشته باشد و تنها تغییرات مفید را از لایه‌ها استخراج کند.

اتصالات باقی‌مانده در بلوک رمزگذار و بلوک رمزگشا

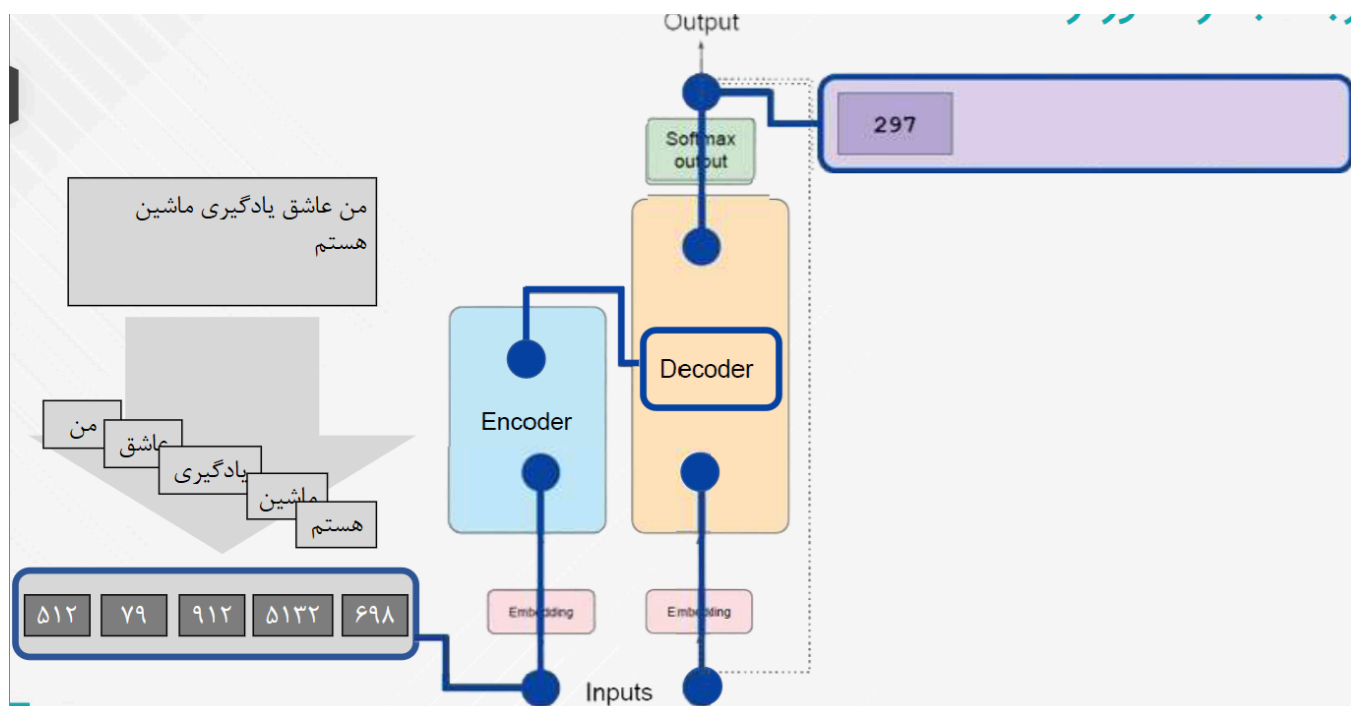
اتصالات باقی‌مانده نه تنها در بلوک رمزگذار (Encoder)، بلکه در بلوک رمزگشا (Decoder) نیز اعمال می‌شوند. در صورتی که مدل دارای دو لایه رمزگذار و دو لایه رمزگشا باشد، ساختار کلی آن به صورت شکل زیر خواهد بود:



بنابراین، هر لایه‌ی توجه به خود و شبکه‌ی عصبی کاملاً متصل (feed forward neural network) در هر رمزگذار (ثدزخیثق) و رمزگشا (decoder)، به کمک اتصالات باقی‌مانده و نرمال‌سازی لایه‌ای خروجی خود را تثبیت کرده و از نابودی اطلاعات در طول پردازش جلوگیری می‌کند.

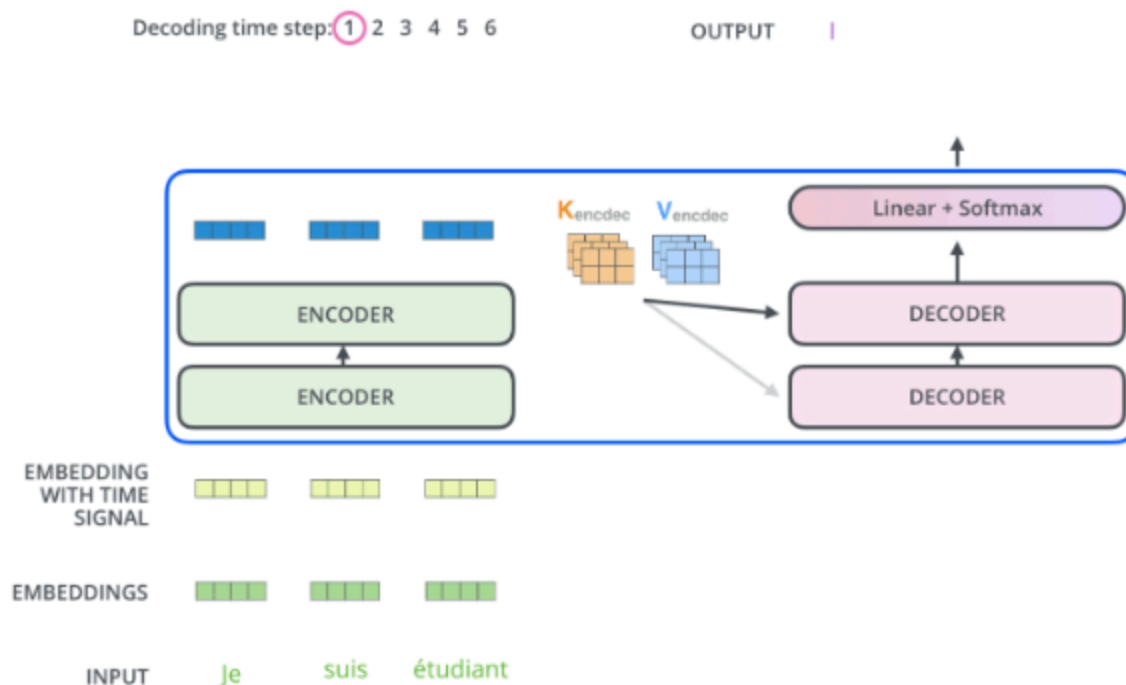
بخش (بلوک) رمزگشا (Decoder) در مدل Transformer

اکنون که بیشتر مفاهیم مرتبط با رمزگذار (Encoder) را بررسی کرده‌ایم، می‌توانیم به تحلیل عملکرد رمزگشا (Decoder) بپردازیم. اگرچه بسیاری از اجزای رمزگشا مشابه رمزگذار هستند، اما عملکرد آن در کنار سایر بخش‌ها نیاز به بررسی دقیق دارد.



عملکرد کلی رمزگشا

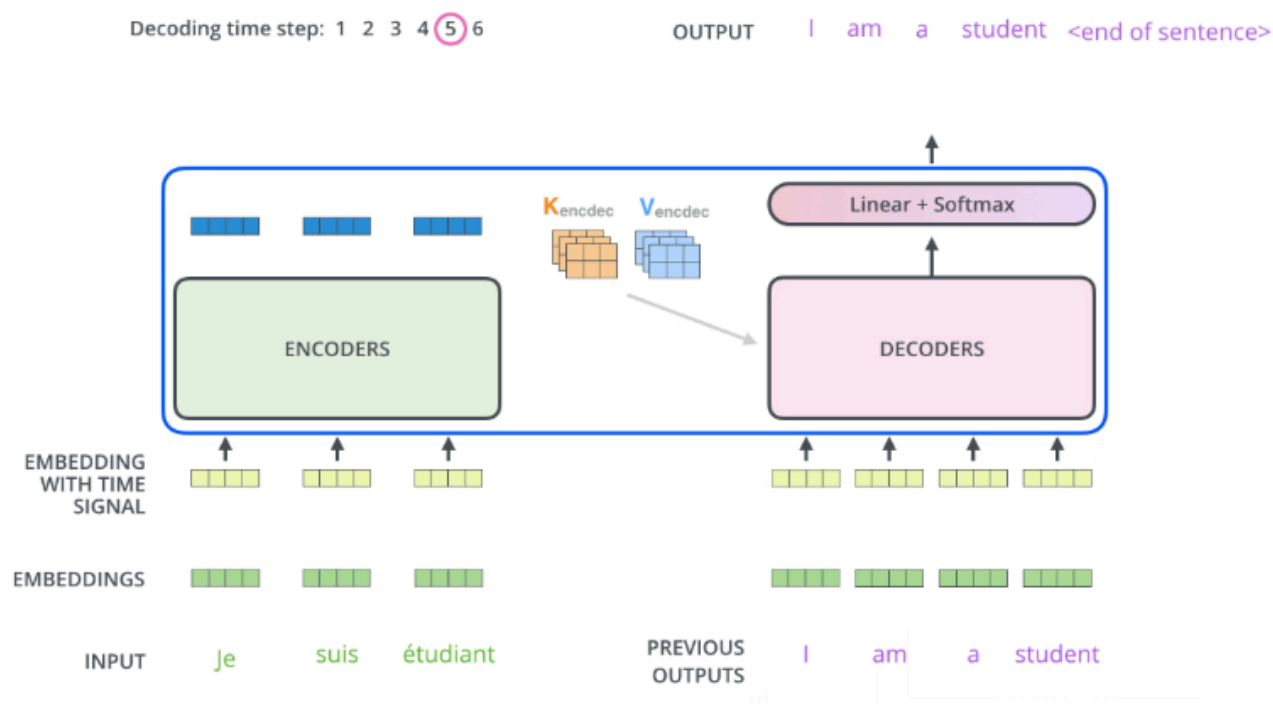
- ابتدا رمزگذار دنباله‌ی ورودی را پردازش کرده و خروجی آن بردارهای توجه K و V را تولید می‌کند.
- این بردارهای توجه به لایه‌ی توجه رمزگذار-رمزگشا (Encoder-Decoder Attention) در رمزگشا منتقل می‌شوند.
- این لایه به تمرکز رمزگشا روی بخش‌های مهم دنباله‌ی ورودی کمک می‌کند و باعث می‌شود مدل در حین تولید خروجی، بتواند اطلاعات کلیدی را از ورودی دریافت کند.



فرآیند رمزگشایی

پس از تکمیل رمزگذاری، فرآیند رمزگشایی آغاز می‌شود. این فرآیند در چندین گام انجام می‌شود:

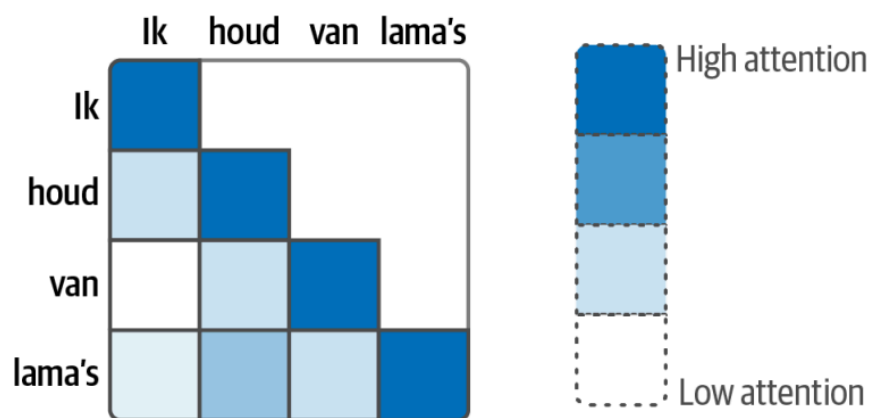
1. رمزگشا یک کلمه از دنباله‌ی خروجی را در هر گام تولید می‌کند.
2. این فرآیند تا رسیدن به یک نماد خاص $\langle \text{EOS} \rangle$ که نشان‌دهنده‌ی پایان دنباله است، تکرار می‌شود.
3. خروجی هر گام، به رمزگشای سطح پایین‌تر در گام بعدی ارسال می‌شود.
4. همانند رمزگذار، به ورودی‌های رمزگشا نیز بردارهای موقعیتی (Positional Encodings) افزوده می‌شود تا موقعیت کلمات در خروجی حفظ شود.



تفاوت‌های توجه در رمزگشا و رمزگذار

لایه‌های توجه به خود (Self-Attention) در رمزگشا عملکردی متفاوت از رمزگذار دارند:

- در رمزگشا، لایه‌ی توجه به خود (Self-Attention) تنها اجازه دارد به موقعیت‌های قبلی در دنباله‌ی خروجی توجه کند.
- این کار با ماسک کردن موقعیت‌های آینده (تنظیم مقدار آن‌ها به $-\infty$) قبل از مرحله‌ی Softmax انجام می‌شود.
- این تکنیک باعث می‌شود که مدل در هر گام فقط از اطلاعات قبلی استفاده کند و از پیش‌گویی‌های غیرمجاز جلوگیری شود.



لایه‌ی توجه رمزگذار-رمزگشا (Encoder-Decoder Attention)

این لایه عملکردی مشابه توجه به خود چندسری (Multi-Headed Self-Attention) دارد اما با یک تفاوت کلیدی:

- ماتریس پرسش‌ها (Queries) از لایه‌ی پایینی رمزگشا دریافت می‌شود.
- ماتریس‌های کلید (Keys) و مقدار (Values) از خروجی رمزگذار استخراج می‌شوند.

لایه‌ی نهایی خطی و Softmax

پشته‌ی رمزگشا (Decoder Stack) در مدل Transformer در نهایت یک بردار از اعداد اعشاری را تولید می‌کند. اما چگونه می‌توان این بردار را به یک کلمه‌ی قابل فهم تبدیل کرد؟ این وظیفه بر عهده‌ی لایه‌ی خطی نهایی (Final Linear Layer) و لایه‌ی Softmax است.

لایه‌ی خطی نهایی (Final Linear Layer)

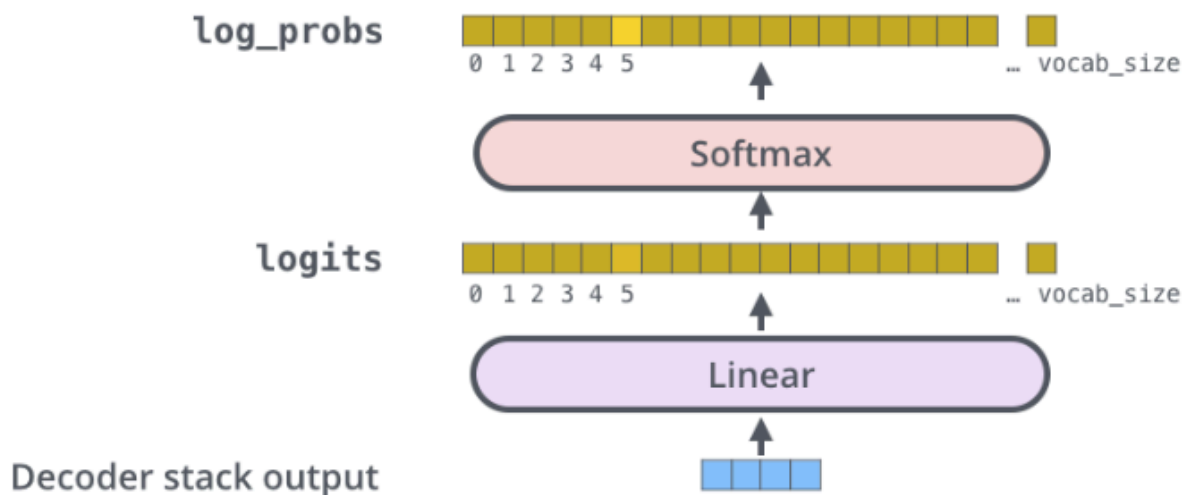
- لایه‌ی خطی یک شبکه‌ی عصبی کاملاً متصل (Fully Connected Neural Network) است.
- این لایه بردار خروجی رمزگشا را به یک بردار بسیار بزرگ به نام `logits` تبدیل می‌کند.
- فرض کنید که مدل ما ۱۰۰۰۰ کلمه‌ی منحصر به فرد انگلیسی را یاد گرفته است.
- در این صورت، بردار `logits` دارای ۱۰۰۰۰ مقدار عددی خواهد بود که هر مقدار نشان‌دهنده‌ی نمره‌ی احتمالی یک کلمه در واژگان مدل است.

Which word in our vocabulary
is associated with this index?

am

Get the index of the cell
with the highest value
(argmax)

5



بردار `logits` شامل نمره‌ی احتمالی ۱۰۰۰۰ کلمه از واژگان مدل است.

لایه‌ی Softmax

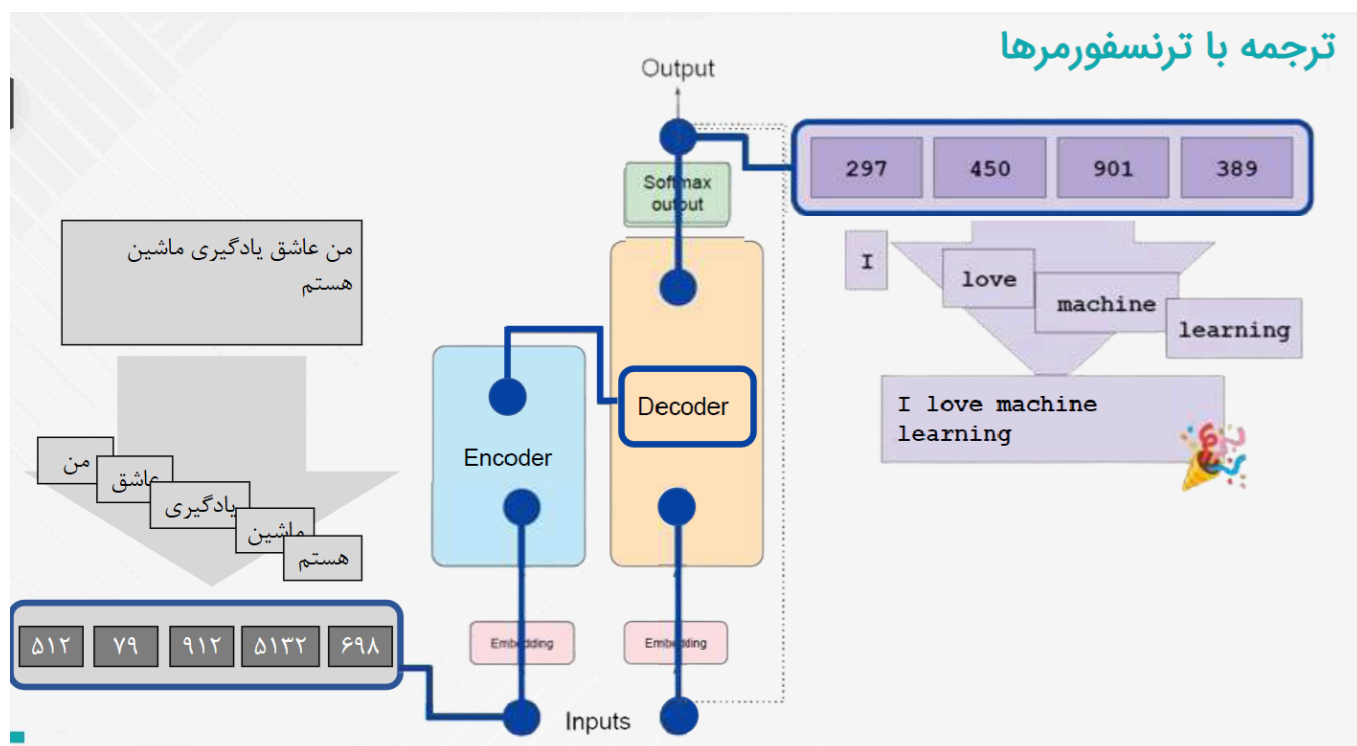
پس از محاسبه‌ی بردار `logits`، این مقادیر باید به احتمال‌هایی قابل تفسیر تبدیل شوند. این کار را لایه‌ی `Softmax` انجام می‌دهد:

- این لایه مقادیر را نرمال‌سازی می‌کند تا همه‌ی آن‌ها اعدادی مثبت باشند و مجموع‌شان برابر ۱ شود.
- به این ترتیب، هر مقدار در بردار `logits` به احتمال تعلق آن کلمه به خروجی مدل تبدیل می‌شود.
- در نهایت، کلمه‌ای که بالاترین احتمال را دارد، به‌عنوان خروجی انتخاب می‌شود.

به این ترتیب، در هر گام تولید متن، مدل بردار خروجی رمزگشا را پردازش کرده، آن را به بردار `logits` تبدیل کرده، از آن احتمال استخراج کرده و در نهایت کلمه‌ی مناسب را انتخاب می‌کند. این فرایند تا رسیدن به نشانه‌ی پایان جمله `<EOS>` تکرار می‌شود.

مروری بر فرآیند آموزش مدل

اکنون که تمامی مراحل گذر رو به جلو (Forward Pass) در یک مدل ترنسفورمر آموزش‌دیده را بررسی کردیم، مفید خواهد بود که مفهوم کلی فرآیند آموزش را نیز مرور کنیم.



نحوه‌ی عملکرد مدل در زمان آموزش

- در طی آموزش، مدل ابتدا بدون هیچ دانشی آغاز به کار می‌کند.
- مدل همان مسیر گذر رو به جلو را طی می‌کند، اما نتیجه‌ی خروجی آن را با مقدار واقعی و درست مقایسه می‌کنیم.
- این مقایسه به مدل امکان می‌دهد تا به‌مرور خطای خود را اصلاح کرده و دقت خروجی‌ها را افزایش دهد.

برای ساده‌سازی، فرض کنیم که واژگان خروجی مدل تنها شامل شش کلمه‌ی زیر است:

"a", "am", "i", "thanks", "student", "<eos>"

که در اینجا، `<eos>` مخفف "پایان جمله" است.

نمایش کلمات به صورت بردارهای یک نقطه‌ای (One-Hot Encoding)

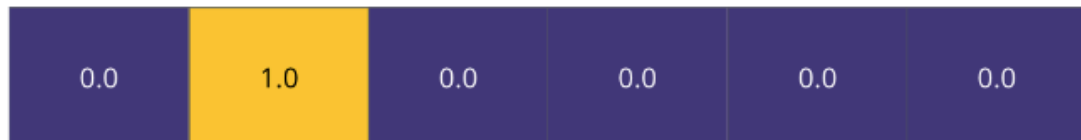
واژگان خروجی مدل در مرحله‌ی پیش‌پردازش و قبل از شروع آموزش تعیین می‌شود. پس از تعریف این واژگان:

- هر کلمه را می‌توان به صورت یک بردار با همان تعداد عناصر نمایش داد.
- این روش نمایش "کدگذاری یک نقطه‌ای (One-Hot Encoding)" نام دارد.
- برای نمونه، کلمه‌ی "am" را می‌توان با بردار زیر نمایش داد:

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"



نمایش کلمات خروجی مدل به صورت One-Hot Encoding.

محاسبه‌ی خطای مدل: تابع هزینه (Loss Function)

پس از مروری بر فرآیند آموزش، نوبت به بررسی تابع هزینه (Loss Function) می‌رسد. تابع هزینه:

- معیاری است که مدل در طول آموزش برای بهینه‌سازی عملکرد خود از آن استفاده می‌کند.
- هدف از بهینه‌سازی کاهش مقدار این تابع و افزایش دقت مدل است.
- در ادامه، تابع هزینه‌ی مدل را با جزئیات بررسی خواهیم کرد.

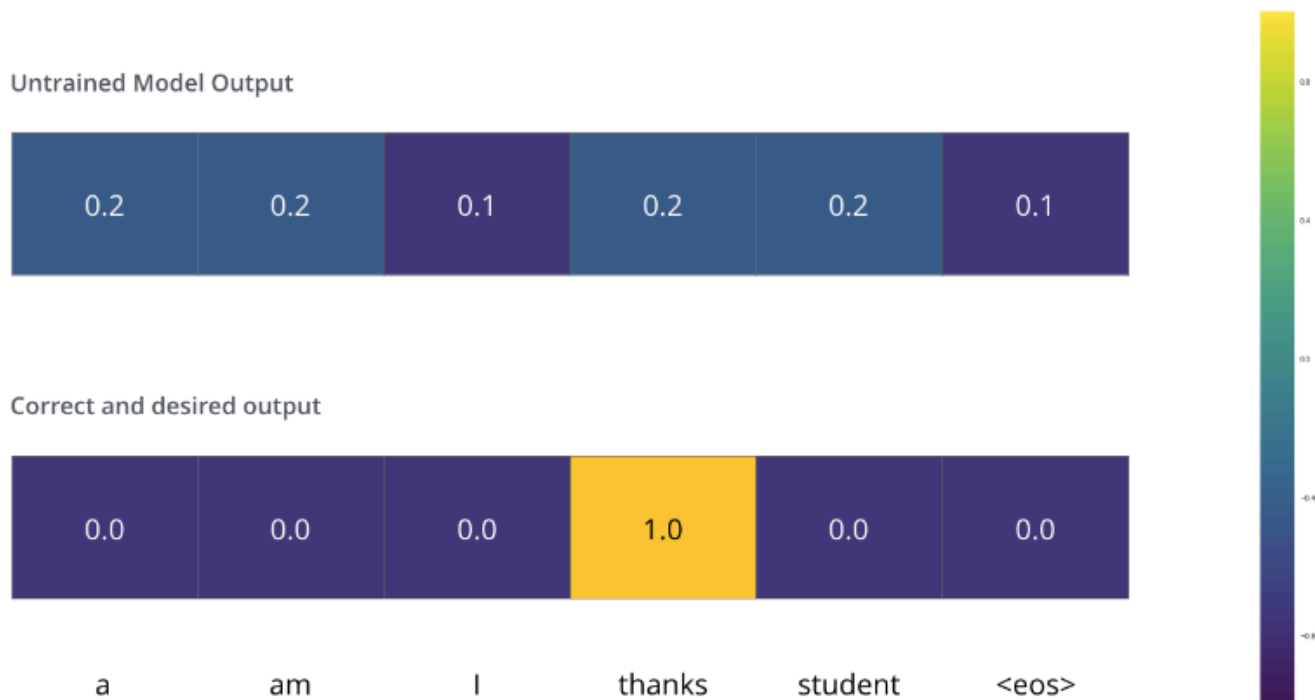
تابع هزینه (Loss Function)

در فرآیند آموزش مدل، هدف بهینه‌سازی عملکرد مدل با کمینه کردن تابع هزینه است. بیایید این مفهوم را با یک مثال ساده توضیح دهیم.

مقایسه‌ی خروجی مدل با مقدار مطلوب

فرض کنید در اولین مرحله‌ی آموزش، مدل را برای ترجمه‌ی عبارت "merci" به "thanks" آموزش می‌دهیم.

- انتظار داریم خروجی مدل یک توزیع احتمالاتی باشد که بیشترین مقدار آن مربوط به کلمه‌ی "thanks" باشد.
- اما از آنجا که مدل هنوز آموزش ندیده است، وزن‌های آن مقداردهی اولیه و تصادفی دارند.
- بنابراین، خروجی اولیه‌ی مدل یک توزیع احتمال تصادفی خواهد بود که احتمالاً مقدار درستی را ارائه نمی‌دهد.
- برای بهبود عملکرد، باید این خروجی را با مقدار صحیح مقایسه کرده و وزن‌های مدل را از طریق پس‌انتشار خطا (Backpropagation) اصلاح کنیم.



خروجی احتمالاتی (logits) ترنسفورمر.

مقایسه‌ی دو توزیع احتمالاتی

برای مقایسه‌ی توزیع احتمال خروجی مدل با توزیع احتمال مطلوب (چون مدل زبانی در واقع یک توزیع احتمالی از کلمات است)، از معیارهای اندازه‌گیری خطا استفاده می‌کنیم:

- آنتروپی متقاطع (Cross-Entropy)
- واگرایی کولبک-لایبلر (Kullback–Leibler Divergence)

این معیارها کمک می‌کنند تا مدل یاد بگیرد که احتمال درستی را برای هر کلمه اختصاص دهد.

مثالی واقعی‌تر: ترجمه‌ی جملات کامل

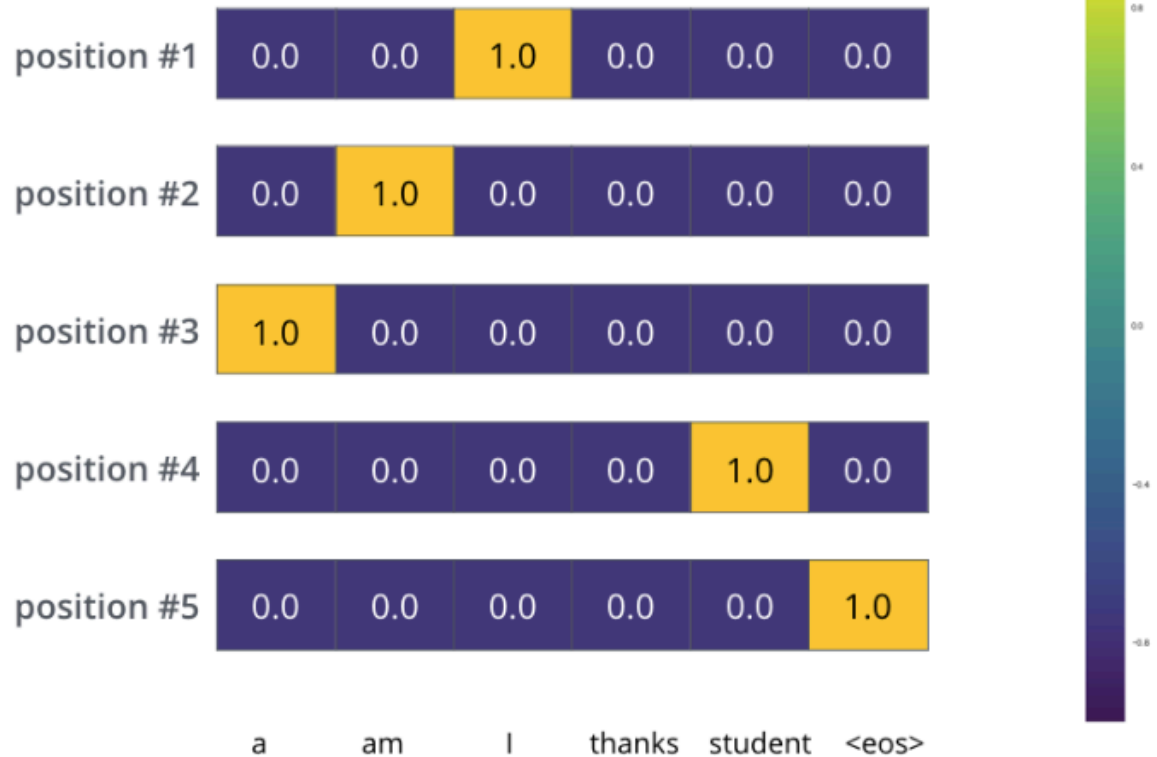
در عمل، مدل نه تنها برای کلمات منفرد بلکه برای جملات کامل آموزش داده می‌شود. به‌عنوان مثال، اگر ورودی مدل عبارت "je suis étudiant" باشد، خروجی مطلوب "I am a student" خواهد بود.

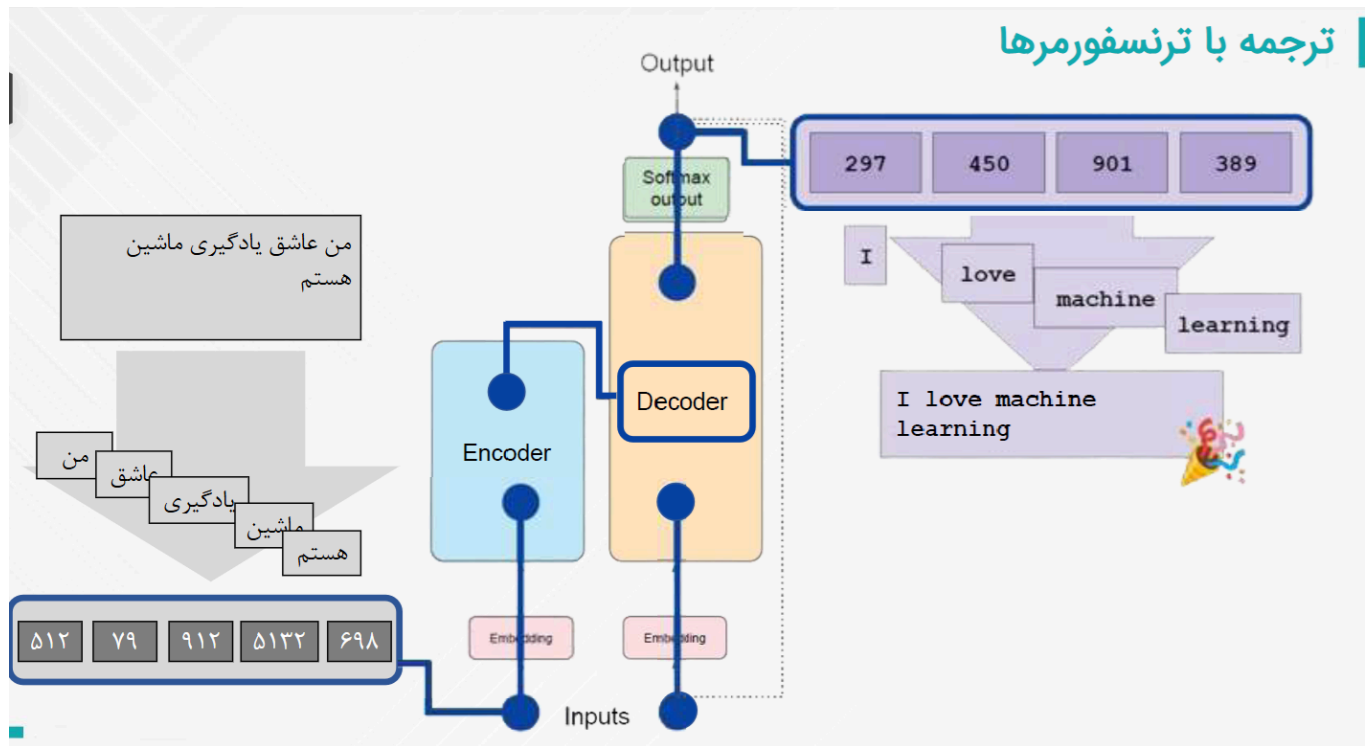
در این حالت، مدل باید چندین توزیع احتمال متوالی تولید کند:

- توزیع احتمال اول باید بیشترین مقدار را در سلول مربوط به کلمه‌ی "I" داشته باشد.
- توزیع احتمال دوم باید بیشترین مقدار را در سلول مربوط به "am" داشته باشد.
- این روند ادامه دارد تا زمانی که به نماد "پایان جمله" (<eos>) برسد.

Target Model Outputs

Output Vocabulary: a am I thanks student <eos>





توزیع های احتمالاتی خروجی یک جمله در مدل ترنسفورمر.

تأثیر آموزش بر بهبود توزیع های خروجی

پس از آموزش مدل با یک مجموعه دادهی بزرگ:

- انتظار داریم که توزیع های احتمال خروجی مدل به مقدار مطلوب نزدیک تر شوند.
- البته، این تضمین نمی کند که مدل همیشه ترجمه ی دقیقی ارائه دهد، به ویژه اگر جمله ی موردنظر در داده های آموزشی وجود نداشته باشد.
- نکته ی مهم: حتی در خروجی های نادرست، مدل همچنان برای تمامی کلمات، احتمال هایی اختصاص می دهد. این ویژگی خاصیت کلیدی تابع Softmax است که روند آموزش را تسهیل می کند.

اهمیت تعمیم دهی مدل

لازم به ذکر است که تولید ترجمه ی صحیح برای یک جمله ی خاص، لزوماً نشان دهنده ی عملکرد کلی مدل نیست. این مسئله به چند دلیل اهمیت دارد:

- ممکن است این جمله قبلاً در داده های آموزشی دیده شده باشد، بنابراین مدل فقط آن را حفظ کرده باشد.
- برای بررسی توانایی تعمیم مدل، باید داده هایی را آزمایش کنیم که در مجموعه ی آموزشی حضور نداشته اند. این امر با استفاده از اعتبارسنجی متقاطع (Cross Validation) انجام می شود.

نقش تابع Softmax در بهبود آموزش

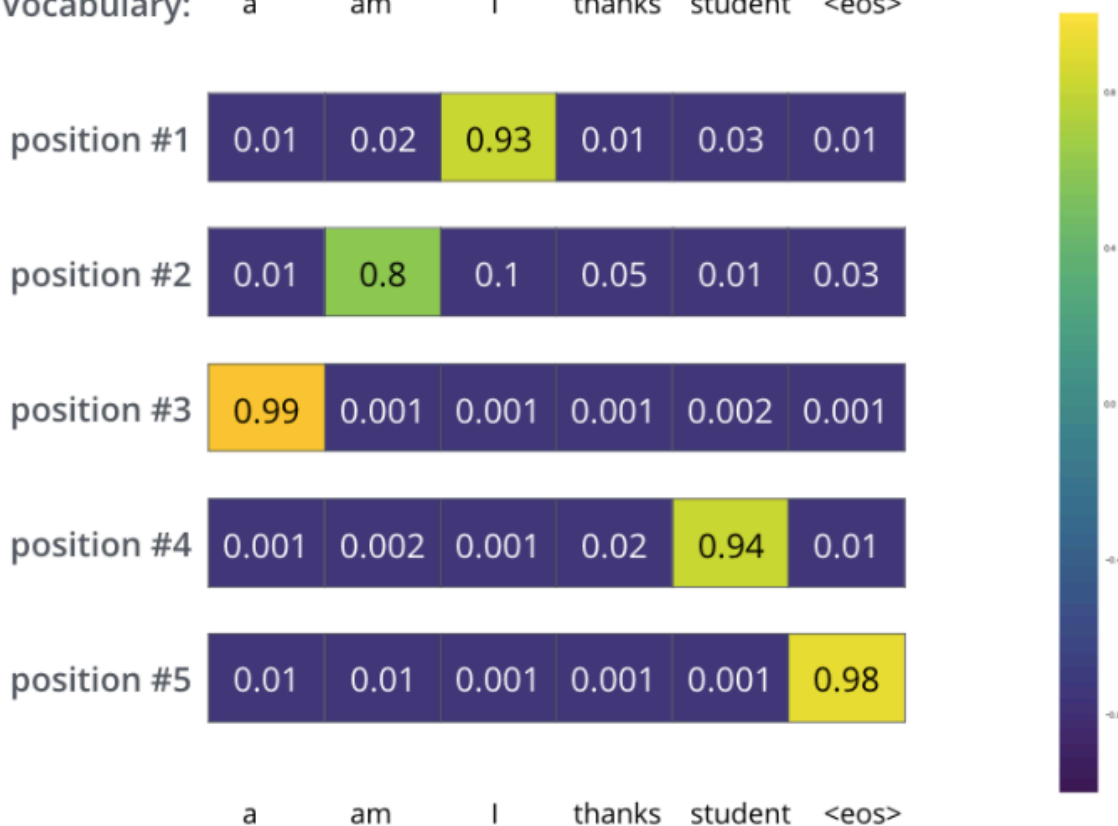
یکی از ویژگی های مفید لایه ی Softmax در فرآیند یادگیری این است که:

- حتی اگر یک کلمه در یک مرحله‌ی خاص احتمال پاییینی داشته باشد، همچنان مقداری از احتمال به آن اختصاص می‌یابد.
- این امر باعث می‌شود که مدل به‌مرور زمان از داده‌های بیشتر یاد بگیرد و توزیع احتمال را بهینه‌تر تنظیم کند.

بنابراین، Softmax نه تنها به انتخاب بهترین خروجی کمک می‌کند، بلکه در طی آموزش باعث هموارسازی گرادیان‌ها و پایداری فرآیند یادگیری می‌شود.

Trained Model Outputs

Output Vocabulary: a am I thanks student <eos>



خروجی مدل آموزش داده شده.

انتخاب بهترین کلمه: روش‌های Greedy Decoding و Beam Search

پارامترهای پیکربندی تولیدی: greedy و random sampling

احتمالات	کلمات
0.20	کیک
0.10	پیراشکی
0.02	موز
0.01	سیب
...	...

حریصانه (Greedy): کلمه/توکنی که بالاترین احتمال را دارد انتخاب می‌شود.

احتمالات	کلمات
0.20	کیک
0.10	پیراشکی
0.02	موز
0.01	سیب
...	...

sampling Random یا نمونه‌برداری تصادفی (وزن‌دار): انتخاب یک توکن با استفاده از یک استراتژی وزن‌دار تصادفی بر اساس احتمالات تمام توکن‌ها. در اینجا، ۲۰٪ احتمال دارد که «کیک» انتخاب شود، اما در واقع «موز» انتخاب شده است.

مدل کلمات خروجی را به صورت گام‌به‌گام تولید می‌کند. انتخاب هر کلمه‌ی جدید می‌تواند به روش‌های مختلفی انجام شود:

۱. روش حریصانه (Greedy Decoding):

- در هر مرحله، مدل فقط کلمه‌ای را انتخاب می‌کند که بیشترین احتمال را دارد.
- این روش سریع است، اما همیشه بهترین جمله‌ی ممکن را تولید نمی‌کند.

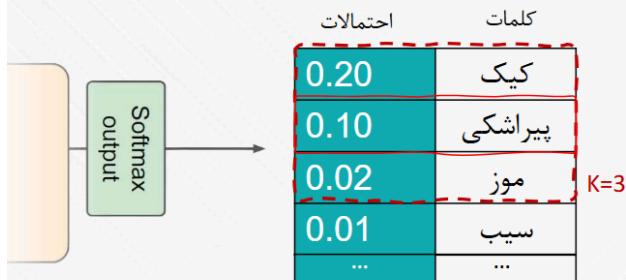
۲. روش جستجوی پرتو (Beam Search):

- به جای انتخاب تنها یک کلمه‌ی با بالاترین احتمال، مدل چندین گزینه را نگه می‌دارد.
- در هر مرحله، مدل چندین جمله‌ی ناقص را ارزیابی می‌کند و بهترین گزینه‌ها را برای ادامه نگه می‌دارد.
- این روش از دو ابرپارامتر (Hyperparameters) استفاده می‌کند:
 - Beam Size:** تعداد جملات ناقص نگه‌داشته‌شده (مثلاً ۲ یا ۵).
 - Top Beams:** تعداد جملاتی که در نهایت به عنوان خروجی ارائه می‌شوند.

مزیت روش Beam Search این است که در جملات طولانی، دقت خروجی مدل بهبود می‌یابد زیرا مدل انتخاب‌های بیشتری را بررسی می‌کند.

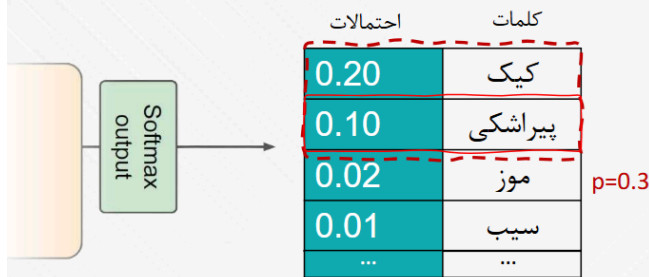
در نهایت، روش انتخابی به نیاز کاربردی مدل بستگی دارد. در بسیاری از کاربردهای واقعی، روش Beam Search عملکرد بهتری نسبت به روش حریصانه دارد.

پارامترهای تولیدی: نمونه برداری top-k



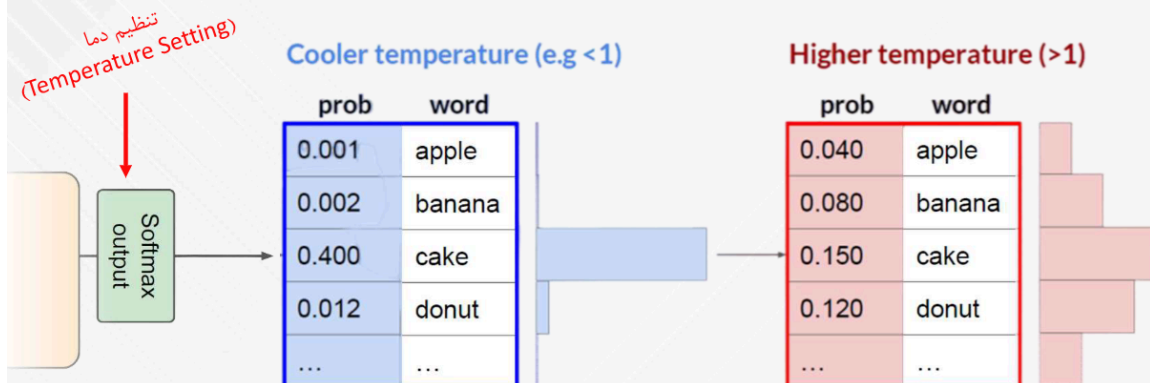
top-k: انتخاب یک خروجی از بین بهترین k نتیجه پس از اعمال استراتژی وزن دار تصادفی با استفاده از احتمالات.

پارامترهای تولیدی: نمونه برداری top-p



top-p: انتخاب یک خروجی با استفاده از استراتژی وزن دار تصادفی از میان نتایج برتر متوالی بر اساس احتمال، به طوری که مجموع احتمالات آن‌ها کمتر یا مساوی p باشد.

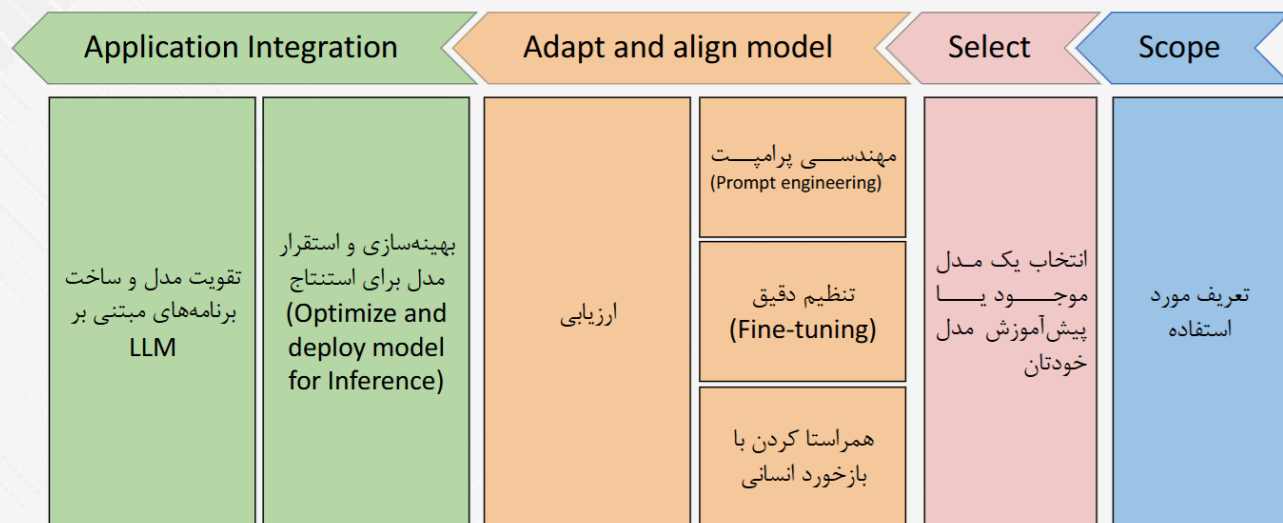
پارامترهای تولیدی: دما یا temperature



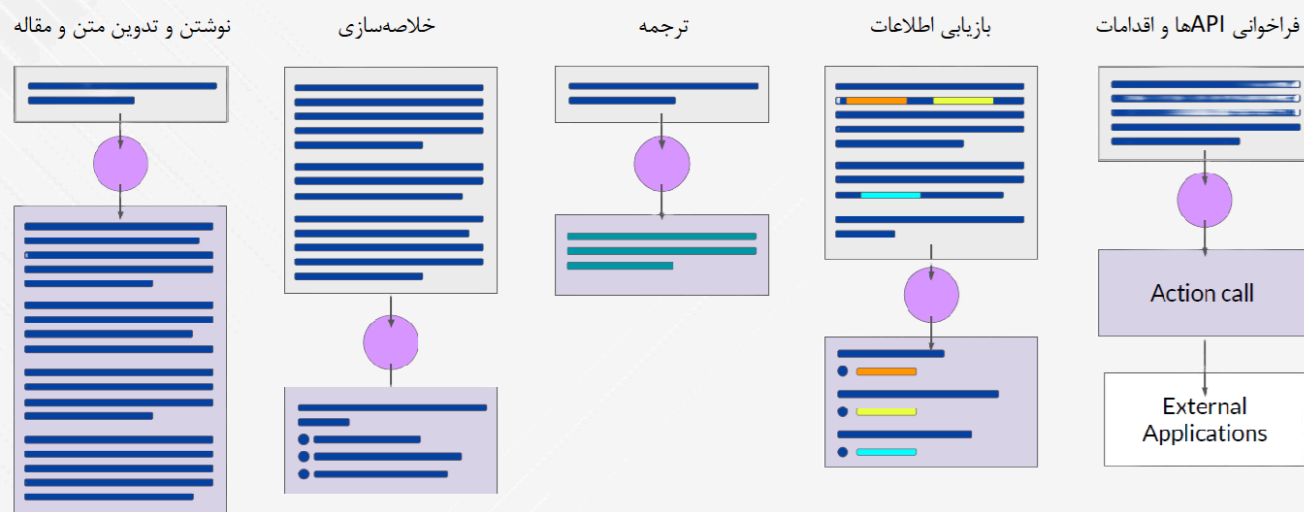
توزیع احتمال با قله‌های شدید (متمرکز)

توزیع احتمال یکنواخت‌تر

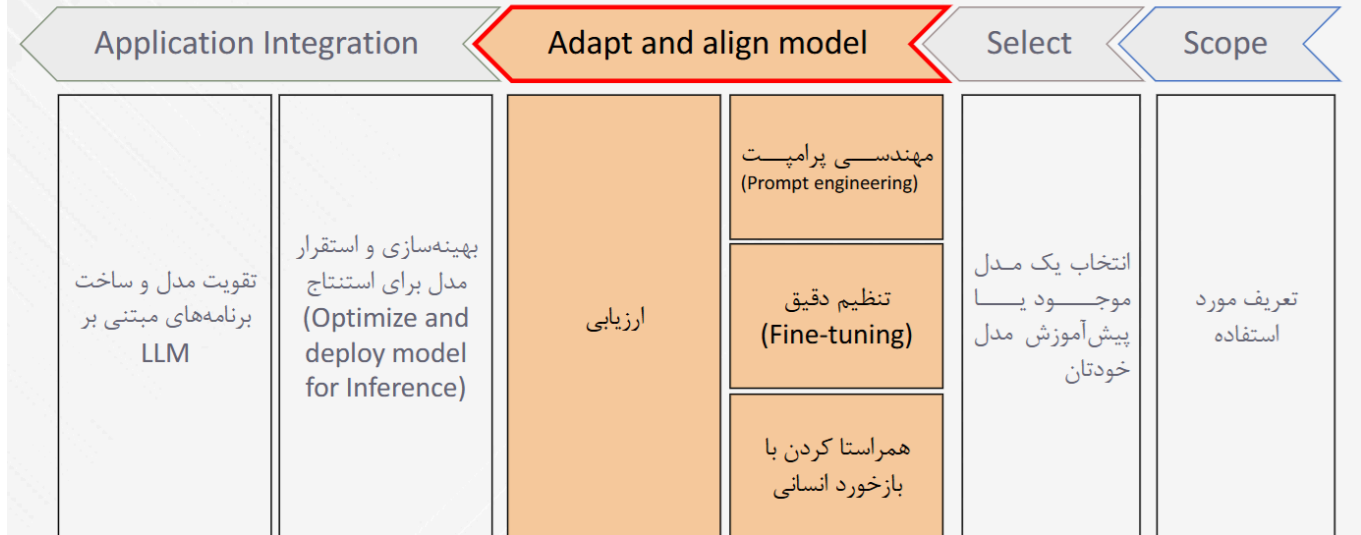
چرخه حیات پروژه Generative AI (هوش مصنوعی مولد)



انجام وظایف متعدد...



چرخه حیات پروژه Generative AI (هوش مصنوعی مولد)



"Pasted image 20251228172004.png1" could not be found.

"Pasted image 20251228172009.png1" could not be found.

"Pasted image 20251228172014.png1" could not be found.

"Pasted image 20251228172019.png1" could not be found.

"Pasted image 20251228172026.png1" could not be found.

"Pasted image 20251228172032.png1" could not be found.

"Pasted image 20251228172038.png1" could not be found.

"Pasted image 20251228172044.png1" could not be found.

"Pasted image 20251228172051.png1" could not be found.

"Pasted image 20251228172057.png1" could not be found.

"Pasted image 20251228172105.png1" could not be found.

"Pasted image 20251228172111.png1" could not be found.

"Pasted image 20251228172118.png1" could not be found.

"Pasted image 20251228172125.png1" could not be found.

"Pasted image 20251228172132.png1" could not be found.